

EEG-BASED EMOTION DETECTION USING ROBERTS SIMILARITY AND PSO FEATURE SELECTION

A project report submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

by

TANJIREDDY DEEPTHI REDDY

22BFA05395

DASARAJU SIVA KARTHIK

22BFA05349

S. A. SADIYA THASNEEM

22BFA05374

VINAY SANGARAJU TIPPARAJU

22BFA05378

Under the guidance of

B. Vijaya, M-Tech, (Ph.D)

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SRI VENKATESWARA COLLEGE OF ENGINEERING
(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu

Accredited by NBA, New Delhi & NAAC with 'A' grade)

Karakambadi Road, TIRUPATI – 517507

2022 - 2026

SRI VENKATESWARA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Ananthapuramu
Accredited by NBA, New Delhi & NAAC with 'A' Grade)
Karakambadi Road, TIRUPATI – 517507.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project work entitled, “ **EEG-BASED EMOTION DETECTION USING ROBERTS SIMILARITY AND PSO FEATURE SELECTION** ” is a bonafide record of the project work done and submitted by

T. DEEPTHIREDDY

22BFA05395

D. SIVA KARTHIK

22BFA05349

S. A. SADHIYA THASNEEM

22BFA05374

S. T. VINAY

22BFA05378

under my guidance and supervision for the partial fulfillment of the requirements for the award of B. Tech degree in **COMPUTER SCIENCE AND ENGINEERING**. This project is the result of our own effort and that it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above

GUIDE

B. Vijaya., M-Tech, (Ph.D)
Assistant Professor
Department of CSE,
SV College of Engineering,
Tirupati.

HEAD OF THE DEPARTMENT

Dr. A. Ganesh., B.Tech., M-Tech., Ph.D (PDF)
Professor and HOD,
Department of CSE,
SV College of Engineering,
Tirupati.

External Viva-Voce Exam held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the project entitled “**EEG-BASED EMOTION DETECTION USING ROBERTS SIMILARITY AND PSO FEATURE SELECTION**”, done by us under the esteemed guidance of **B. VIJAYA**, Assistant Professor, and is submitted in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering**.

This report is the result of our own effort and it has not been submitted to any other University or Institution for the award of any degree or diploma other than specified above.

TANJIREDDY DEEPTHIREDDY	22BFA05395
DASARAJU SIVA KARTHIK	22BFA05349
S. A. SADHIYA THASNEEM	22BFA05374
VINAY SANGARAJU TIPPARAJU	22BFA05378

Date:

Place:

ACKNOWLEDGEMENT

We are thankful to our guide **B. Vijaya, M-Tech, (Ph.D), Assistant Professor, Department of CSE** for her valuable guidance and encouragement. Her helping attitude and suggestions have helped in the successful completion of the project report.

We would like to express our gratefulness and sincere thanks to **Dr. A. Ganesh., Ph.D (PDF), Professor & HOD, Department of CSE**, for his kind help and encouragement during the course of study and in the successful completion of the project work.

We have great pleasure in expressing out hearty thanks to our beloved Principal **Dr. Vijaya Gunturu** for spending his valuable time with us to complete this project.

Successful completion of any project cannot be done without proper support and encouragement. We sincerely thank the Management for providing all the necessary facilities during course of study.

We would like to thank our parents and friends, who have the greatest contributions in all our achievements, for the great care and blessings in making us successful in all our endeavors.

TANJIREDDY DEEPTHI REDDY

22BFA05395

DASARAJU SIVA KARTHIK

22BFA05349

S. A. SADHIYA THASNEEM

22BFA05374

VINAY SANGARAJU TIPPARAJU

22BFA05378

ABSTRACT

The detection of emotions through electroencephalography (EEG) signals has gained significant attention due to its potential applications in areas such as mental health diagnosis, human-computer interaction, and affective computing. This work proposes a novel emotion detection framework that integrates EEG-based signal processing with advanced machine learning and deep learning techniques. To enhance feature extraction, an Extra Trees Classifier is utilized to identify the most relevant features from EEG signals, which are then optimized using the PSO Feature Model Selector. The feature selection model employs a combination of K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Random Forest (RF) to refine the selection process and improve classification accuracy.

For the classification task, the study leverages both traditional machine learning methods and deep learning architectures. A hybrid CNN-LSTM model is utilized, which effectively captures spatial and temporal dependencies in EEG data. Additionally, a Roberta Similarity-based approach is integrated to quantify the similarity between EEG signal patterns, improving the overall emotion classification process. The framework also explores the use of DeepConvNet, a convolutional neural network-based model, for extracting hierarchical features from raw EEG data, enhancing the detection of complex emotional states.

The results demonstrate the efficacy of combining traditional machine learning models with cutting-edge deep learning architectures, highlighting improvements in accuracy, robustness, and real-time processing capabilities for emotion detection from EEG signals. This work contributes to advancing emotion-aware systems that can interact with users in a more personalized and empathetic manner, paving the way for future applications in healthcare, entertainment, and human-machine interaction.

CONTENTS

	Abstract	i
	Table of contents	ii
	List of figures	iii
	List of tables	iv
	Abbreviations	v
Chapter No.	Description	Page No.
1	INTRODUCTION	01
2	PROJECT DESCRIPTION	02
	2.1 PROBLEM DEFINITION	02
	2.2 PROJECT DETAILS	03
3	COMPUTATIONAL ENVIROMENT	04
	3.1 SOFTWARE REQUIREMENTS	04
	3.2 HARDWARE REQUIREMENTS	04
	3.3 FUNCTIONAL REQUIREMENTS	05
	3.4 SOFTWARE FEATURES	06
4	FEASIBILITY STUDY	09
	4.1 TECHNICAL FEASIBILITY	10
	4.2 OPERATIONAL FEASIBILITY	10
	4.3 ECONOMICAL FEASIBILITY	10
5	SYSTEM ANALYSIS	12
	5.1 EXISTING SYSTEM	12
	5.1.1 Disadvantages of Existing System	12
	5.2 PROPOSED SYSTEM	13
	5.2.1 Advantages of Proposed System	14
6	SYSTEM DESIGN	16
	6.1 SYSTEM ARCHITECTURE	17
	6.2 UML DIAGRAMS	21

Chapter No.	Description	Page No.
7	SYSTEM IMPLEMENTATION	29
	7.1 ALGORITHM	29
	7.2 WORKING AND MODULE EXPLANATION	33
8	SOURCE CODE	35
9	TESTING	43
	9.1 UNIT TESTING	43
	9.2 INTEGRATION TESTING	44
	9.3 FUNCTIONAL TESTING	45
	9.4 SYSTEM TESTING	46
	9.5 ACCEPTANCE TESTING	47
	9.6 BLACK BOX TESTING	48
	9.7 WHITE BOX TESTING	49
10	ALGORITHM PERFORMANCE RESULTS	50
11	SCREEN LAYOUTS	55
12	CONCLUSION	61
13	FUTURE ENHANCEMENTS	62
14	BIBLIOGRAPHY	63

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
6.1	System Architecure	18
6.2.1	Class Diagram	23
6.2.2	Use Case Diagram	24
6.2.3	Sequence Diagram	25
6.2.4	Activity Diagram	26
6.2.5	Component Diagram	27
6.2.6	Deployment Diagram	28
9.3	System Testing	47
10.1	PSO Classification report	50
10.2	PSO Confusion matrix	51
10.3	CNN-LSTM with Roberta Similarity Classification report	52
10.4	CNN-LSTM with Roberta Similarity Confusion matrix	52
10.5	DeepConvNet Classification report	53
10.6	DeepConvNet Confusion matrix	54
11.1	Landing Home Page	55
11.2	Register Page	56
11.3	Login Page	56
11.4	Dashboard	57
11.5	Algorithm-Performance-Results-1	58
11.6	Algorithm-Performance-Results-2	58
11.7	Prediction-Page-1	59
11.8	Prediction-page-2	60

LIST OF TABLES

Table No.	Table Name	Page No.
8.1	Unit Testing	43
8.2	Integration Testing	45

ABBREVIATIONS

AI	–	Artificial Intelligence
CNN	–	Convolutional Neural Network
DL	–	Deep Learning
EEG	–	Electroencephalography
ETC	–	Extra Trees Classifier
KNN	–	K-Nearest Neighbors
LSTM	–	Long Short-Term Memory
ML	–	Machine Learning
PSO	–	Particle Swarm Optimization
RF	–	Random Forest
RoBERTa	–	Robustly Optimized BERT Approach
SVM	–	Support Vector Machine
UML	–	Unified Modeling Language

Chapter 1

INTRODUCTION

Emotion detection plays a critical role in understanding human behavior and improving the interaction between humans and machines. Electroencephalography (EEG) signals are increasingly being explored for emotion recognition due to their non-invasive nature and ability to reflect neural activity associated with emotional states. However, interpreting emotional states from EEG signals is a challenging task due to the complexity, high dimensionality, and inherent noise in the data.

This project aims to develop an advanced emotion detection system using EEG signals by integrating multiple machine learning and deep learning techniques to enhance classification accuracy and efficiency. The core of the system involves feature extraction using an Extra Trees Classifier, which efficiently identifies the most relevant features from EEG signals. These features are further optimized using the PSOFeatureModelSelector, which employs Particle Swarm Optimization (PSO) to refine the selection process. The feature selection process leverages models such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Random Forest (RF) to improve the model's performance.

For the emotion classification task, the project incorporates a hybrid CNN-LSTM model, which captures both spatial and temporal patterns in the EEG data. This allows the model to better understand the complex nature of emotional responses, improving its classification ability. Additionally, Roberta Similarity is integrated to assess the similarity between EEG signal patterns, further refining the classification process. The DeepConvNet model, which utilizes convolutional neural networks, is also explored to extract hierarchical features directly from the raw EEG data.

The proposed system is expected to improve emotion detection performance in real-time applications, offering potential benefits in areas like mental health diagnosis, human-computer interaction, and affective computing.

Chapter 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION

Emotion detection using Electroencephalography (EEG) signals is an important area in fields such as healthcare, human–computer interaction, and mental health monitoring. EEG signals represent electrical brain activity and help in understanding human emotional states. However, detecting emotions from EEG signals is a challenging task because the signals are complex, high-dimensional, and often contain noise.

Traditional emotion detection methods mainly rely on basic feature extraction and simple machine learning algorithms. These approaches often fail to capture the complex patterns present in EEG signals. As a result, they may produce low accuracy and unreliable emotion classification results. Another limitation is that many existing systems cannot effectively capture both spatial and temporal relationships in EEG data, which are important for accurate emotion recognition.

In addition, many current systems do not support real-time emotion detection. Real-time analysis is important for applications such as mental health monitoring, adaptive interfaces, and emotion-aware computing systems. Without efficient feature extraction and classification methods, it becomes difficult to build reliable emotion detection systems.

Therefore, there is a need for an improved system that can handle complex EEG signals and accurately classify emotions. This project aims to address these challenges by using advanced feature extraction techniques, optimized feature selection, and powerful machine learning and deep learning models to improve emotion detection accuracy and performance.

2.2 PROJECT DETAILS:

The proposed project focuses on developing an EEG-based emotion detection system

using machine learning and deep learning techniques. The system analyzes EEG signals identify different emotional states by extracting meaningful features from brain activity data.

Initially, EEG signals are processed using the Extra Trees Classifier to extract important features from the raw data. This helps reduce noise and identify the most relevant features required for emotion classification. After feature extraction, Particle Swarm Optimization (PSO) is used through the PSO Feature Model Selector to select the best subset of features for improving model performance.

The selected features are then used for emotion classification using machine learning algorithms such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Random Forest (RF). These algorithms help classify emotional states based on EEG signal patterns.

In addition, a CNN-LSTM hybrid deep learning model is implemented to capture both spatial and temporal patterns in EEG signals. This improves the system's ability to understand complex emotional responses. The project also incorporates Roberta Similarity to measure similarities between EEG signal patterns and improve classification accuracy.

The developed system aims to provide accurate and efficient emotion detection from EEG signals. The results of this system can be useful in applications such as healthcare monitoring, human-computer interaction, and emotion-aware intelligent systems.

Chapter 3

COMPUTATIONAL ENVIRONMENT

3.1 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating costs, planning team activities, performing tasks and tracking the team's and tracking the team's progress throughout the development activity.

Operating System	: Windows 7/8/10
Server side Script	: HTML, CSS, Bootstrap & JS
Programming Language	: Python
Libraries	: Flask, Pandas, Torch, Sklearn, Librosa, Numpy , Seaborn, Matplotlib
IDE/Workbench	: VSCode
Server Deployment	: Xampp Server
Database	: MySQL

3.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It shows what the system does and not how it should be implemented. Ambiguities or omissions in these requirements can lead to significant cost overruns and delays during the project lifecycle.

Processor	- I3/Intel Processor
RAM	- 8GB (min)
Hard Disk	- 128 GB

Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- Any

3.3 FUNCTIONAL REQUIREMENTS

The EEG-Based Emotion Detection System is designed to perform important functions for accurate emotion recognition using EEG signals. The system allows users to upload EEG signal data, which represents brain activity related to different emotional states. This data is securely stored and processed within the system. During preprocessing, the EEG signals are cleaned by removing noise and artifacts, and the data is normalized to improve the quality and accuracy of analysis.

The system integrates machine learning and deep learning models such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Random Forest, CNN-LSTM, and DeepConvNet. These models analyze the processed EEG signals to identify patterns related to emotions. Feature extraction is performed using the Extra Trees Classifier, and feature optimization is done using Particle Swarm Optimization (PSO) to select the most relevant features for better prediction.

Additionally, the system applies RoBERTa Similarity to enhance emotion classification accuracy. The models are trained and tested using EEG datasets, and the system finally predicts and displays the detected emotional state based on the input EEG signals.

EEG Data Input Preprocessing : Allows users to upload EEG signal data and performs noise removal, normalization, and preprocessing to improve data quality.

ML/DL Model Integration: Integrates machine learning and deep learning models such as KNN, SVM, Random Forest, CNN-LSTM, and DeepConvNet to analyze EEG signals and predict emotional states.

3.4 SOFTWARE FEATURES

Python:

Python is a high-level programming language widely used for machine learning, data analysis, and artificial intelligence applications. It is easy to read and write due to its

simple syntax. Python supports multiple programming paradigms such as procedural, object-oriented, and functional programming.

In this project, Python is used to develop the core functionality of the EEG-based emotion detection system. It helps in data preprocessing, feature extraction, model training, and prediction. Python also supports various libraries that simplify complex computations and machine learning tasks.

Advantages:

- i. Simple and Easy to Learn:** Python has a clear syntax that makes it easy for beginners to understand and develop programs.
- ii. Large Standard Library:** Python provides many built-in libraries and frameworks useful for data analysis, machine learning, and visualization.
- iii. Cross-Platform Compatibility:** Python programs can run on multiple operating systems such as Windows, Linux, and macOS.
- iv. Rapid Development:** Python allows quick development and testing of applications.
- v. High-Level Language:** It reduces the complexity of programming by allowing developers to write fewer lines of code.
- vi. Extensible:** Python supports integration with powerful libraries such as NumPy, Pandas, Torch, and Scikit-learn.

Flask

Flask is a lightweight Python web framework used to develop web-based applications. It provides simple tools and libraries to build web interfaces and connect the backend machine learning models with the user interface.

In this project, Flask is used to create the server-side application that processes user inputs, communicates with the machine learning models, and displays the predicted emotion results through a web interface.

Librosa

Librosa is a Python library used for analyzing and processing signal data. It provides tools for feature extraction and signal analysis.

In this project, Librosa helps process signal data and extract important features that can be used for emotion detection and machine learning analysis.

Torch (PyTorch)

Torch is a powerful deep learning framework used to build and train neural network models. It supports GPU acceleration and provides flexible tools for developing deep learning applications.

In this project, Torch is used to implement deep learning models for analyzing EEG signals and improving the accuracy of emotion detection.

MySQL

MySQL is an open-source relational database management system used to store and manage data efficiently.

In this project, MySQL is used to store user inputs, processed data, and prediction results. It helps manage data securely and ensures quick retrieval when required.

Visual Studio Code:

Visual Studio Code (VS Code) is a free, open-source code editor developed by Microsoft. It provides an extensive set of features and extensions that can be customized to fit the needs of developers across a wide range of programming languages and frameworks. VS Code is a powerful code editor that is highly customizable and provides a range of features and tools to help developers work more efficiently. Apart from performing its core tasks, the system also includes important software features to enhance performance and usability:

1. **Performance:** Capable of quickly processing large datasets and generating results in real time.
2. **Scalability:** Designed to support multiple users and a growing dataset without performance issues.
3. **Usability:** Simple and user-friendly interface built using Streamlit, making it easy for users to input data and view results.
4. **Maintainability:** Well-structured code and documentation make future updates and debugging easier.
5. **Security:** Strong data protection mechanisms are in place to ensure user privacy and data safety.

Chapter 4

FEASIBILITY STUDY

A feasibility study is conducted to determine whether the proposed system is practical and beneficial before beginning the development process. It helps in understanding whether the project can be successfully implemented using the available resources, time, and technology. During this phase, the major requirements of the system are analyzed to ensure that the development of the project will not cause unnecessary financial, technical, or operational difficulties. The main objective of this study is to evaluate whether the proposed system is worth implementing and whether it will provide useful results.

For the proposed **EEG-Based Emotion Detection System using RoBERTa Similarity and PSO Feature Selection**, the feasibility study is carried out to examine whether the system can effectively process EEG signals and predict emotional states using machine learning techniques. This analysis helps in identifying the required tools, technologies, and system resources needed for implementation. By conducting this study, potential challenges can be identified early and appropriate solutions can be planned.

Three key considerations involved in the feasibility analysis are:

1. Technical Feasibility
2. Operational Feasibility
3. Economical Feasibility

4.1 TECHNICAL FEASIBILITY

Technical feasibility determines whether the required technical resources and technologies are available to successfully develop and implement the system. The proposed emotion detection system relies on widely used technologies such as Python, Flask, machine learning libraries, and signal processing tools.

These technologies are well-documented and supported by large developer communities, making development and troubleshooting easier. The system processes EEG signal data, performs feature extraction and optimization using PSO, and applies machine learning models for emotion classification. All these processes can be executed using existing frameworks and standard computing resources. Since the necessary tools and infrastructure are readily available and the technical requirements are manageable, the proposed system is technically feasible.

4.2 OPERATIONAL FEASIBILITY

Operational feasibility focuses on how well the system will function in the real-world environment and whether users will be able to effectively use it. The proposed system is designed with a simple and user-friendly interface that allows users to upload EEG signal data and obtain emotion prediction results easily.

The system workflow is straightforward, requiring minimal user interaction. Users only need basic knowledge of the system to operate it effectively. Additionally, the system provides clear outputs and visualizations, which help users understand the results easily. Because the system is designed to be easy to use and requires minimal training, it is expected to integrate smoothly into practical applications, making it operationally feasible.

4.3 ECONOMICAL FEASIBILITY

Economic feasibility evaluates whether the proposed system can be developed and maintained within a reasonable budget. The goal of this analysis is to ensure that the benefits of the system justify the costs involved in its development and deployment. In the case of the EEG-Based Emotion Detection System, the overall development cost is relatively low because most of the technologies used are open-source and freely available.

Programming languages such as Python and libraries like NumPy, Pandas, Torch, and Scikit-learn do not require licensing costs. The development environment, Visual Studio Code, is also freely available, which further reduces expenses. Additionally, the system can run on standard computing hardware without requiring expensive specialized

equipment. As a result, the project can be implemented within a limited budget while still delivering effective results. Therefore, the proposed system is considered economically feasible.

Chapter 5

SYSTEM ANALYSIS

5.1 EXISTING SYSTEM

Existing EEG-based emotion detection systems primarily rely on traditional feature extraction methods such as power spectral density, wavelet transform, and statistical measures to capture emotion-related brainwave patterns. These systems use machine learning algorithms like Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Random Forest (RF) for classification. However, they often struggle with high-dimensionality, noise, and limited generalization across diverse emotional states. Some systems integrate deep learning models like Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks, but they still face challenges in achieving high accuracy, real-time processing, and efficient feature selection from EEG signals.

5.1.1 Disadvantages of Existing System

The existing EEG-based emotion detection systems have several limitations:

1. **Accuracy Issues:** Traditional feature extraction methods struggle with high-dimensionality and noise, leading to suboptimal classification accuracy.
2. **Limited Generalization:** These systems often fail to generalize well across different emotional states and individuals, reducing their robustness.
3. **Real-time Processing:** Many existing systems cannot process EEG data in real-time, limiting their practical application in interactive environments.
4. **Feature Selection:** The feature extraction and selection methods are often inefficient, failing to capture the most relevant patterns in EEG signals for emotion recognition.
5. **Complexity:** Some models are computationally expensive, making them unsuitable for low-resource or mobile applications.

6. **Overfitting:** Traditional models may overfit to the training data, affecting their ability to perform accurately on unseen data.

5.2 PROPOSED SYSTEM

The proposed system aims to address the limitations of existing EEG-based emotion detection systems by integrating advanced feature extraction, selection, and classification techniques. Key components of the system include:

1. **Feature Extraction with Extra Trees Classifier:** The system uses the Extra Trees Classifier to efficiently extract relevant features from EEG signals, reducing noise and dimensionality.
2. **Optimized Feature Selection:** The PSO Feature Model Selector, leveraging Particle Swarm Optimization (PSO), refines the feature selection process. It combines multiple classifiers like KNN, SVM, and Random Forest to improve accuracy.
3. **Deep Learning Models:** The system utilizes a hybrid CNN-LSTM model for capturing spatial and temporal dependencies in EEG signals, enhancing classification accuracy. Additionally, the Roberta Similarity model is integrated to assess signal pattern similarities, improving emotion recognition.
4. **Real-Time Processing:** The system is optimized for real-time emotion detection, making it suitable for applications in healthcare, human-computer interaction, and mental health monitoring.
5. **DeepConvNet:** The integration of DeepConvNet further enhances the model's ability to extract hierarchical features directly from raw EEG data, improving overall performance.

The proposed system aims to achieve higher accuracy, real-time processing capabilities, and robust emotion recognition, addressing the challenges of existing systems in terms of feature selection, computational efficiency, and generalization.

5.2.1 Advantages of Proposed System

The proposed EEG-based emotion detection system offers several key advantages over existing systems:

1. **Improved Accuracy:** By utilizing the Extra Trees Classifier for feature extraction and optimizing the feature selection process with PSO, the system ensures more relevant features are selected, leading to higher classification accuracy.
2. **Efficient Feature Selection:** The use of PSOFeatureModelSelector combined with multiple classifiers (KNN, SVM, RF) optimizes feature selection, addressing the challenges of high-dimensionality and noise in EEG signals.
3. **Real-Time Processing:** The system is designed for real-time emotion detection, enabling immediate feedback for applications in healthcare, human-computer interaction, and mental health monitoring.
4. **Hybrid Deep Learning Models:** The integration of CNN-LSTM models allows the system to capture both spatial and temporal dependencies in EEG data, significantly improving emotion recognition accuracy and robustness.
5. **Enhanced Generalization:** The system's ability to generalize across different emotional states and individuals is improved through the use of advanced models like Roberta Similarity and DeepConvNet, leading to more consistent performance.
6. **Scalability:** With efficient feature extraction and selection, the system is scalable and can be adapted for various applications, including wearable devices and interactive environments.
7. **Low Computational Cost:** Compared to traditional methods, the proposed system is optimized for computational efficiency, making it suitable for deployment in resource-constrained environments such as mobile devices and edge computing.

These advantages contribute to the system's effectiveness in delivering accurate, real-time, and scalable emotion detection, making it more applicable for a wide range of real-world applications.

Chapter 6

SYSTEM DESIGN

The system design of the **EEG-Based Emotion Detection System** focuses on developing an efficient framework that can analyze EEG signals and accurately predict emotional states. The design phase involves several important processes such as data collection, preprocessing, feature extraction, model training, and result generation. These steps ensure that the system can process EEG signal data effectively and provide reliable emotion classification results.

Data Collection

The dataset used in this project is obtained from the **Kaggle platform**, which provides publicly available datasets for machine learning and research purposes. The dataset contains **EEG (Electroencephalogram) signals representing brainwave activity recorded from multiple subjects during emotional stimuli**. The signals are labeled with different emotional states such as **Happy, Sad, Neutral, and Angry**. The EEG signals are stored in structured formats such as **CSV or MAT files**, which allows easy data handling and processing.

Data Preprocessing

Raw EEG data may contain noise, missing values, and inconsistencies that can affect the accuracy of the prediction models. Therefore, preprocessing techniques are applied to clean and prepare the data. The EEG signals are normalized to a common range to maintain consistency across different samples. Additional preprocessing steps include noise removal, handling missing values, and preparing the data for further analysis.

Feature Extraction

Feature extraction is an important step in identifying meaningful patterns from EEG signals. Various features are extracted from the signals, including **statistical features such as mean, standard deviation, skewness, and kurtosis**. In addition, **time-**

frequency features derived from EEG frequency bands like **alpha, beta, and delta** are also extracted. These features help represent EEG signals in a structured form suitable for machine learning models.

Dataset Splitting and Model Training

After preprocessing and feature extraction, the dataset is divided into **training and testing sets**. Typically, **80% of the data is used for training** and **20% for testing** the model. Several machine learning models such as **Support Vector Machine (SVM), Random Forest, LightGBM, XGBoost, and Multi-Layer Perceptron (MLP)** are trained using the processed data to classify emotional states.

Model Evaluation and Deployment

The trained models are evaluated using performance metrics such as **accuracy, precision, recall, and F1-score** to measure their effectiveness. After selecting the best-performing model, it is deployed in a **Flask-based web application**. The trained model processes uploaded EEG signals and predicts the emotional state, displaying the results to the user through the web interface.

6.1 SYSTEM ARCHITECTURE

The below figure illustrates the architectural design of the **EEG-Based Emotion Detection System**. This architecture provides a summarized and high-level view of the project, explaining how the system processes EEG signals and predicts emotional states. It highlights the main components of the system and the workflow involved in processing EEG data using machine learning and deep learning techniques.

The system architecture begins with the **collection of EEG signal data** from reliable sources such as the **Kaggle dataset repository**. Once the dataset is collected, it undergoes several preprocessing steps to prepare the data for analysis. The processed data is then used to train machine learning and deep learning models that classify different emotional states. Finally, the trained model is deployed through a web-based application where users can upload EEG data and receive emotion prediction results.

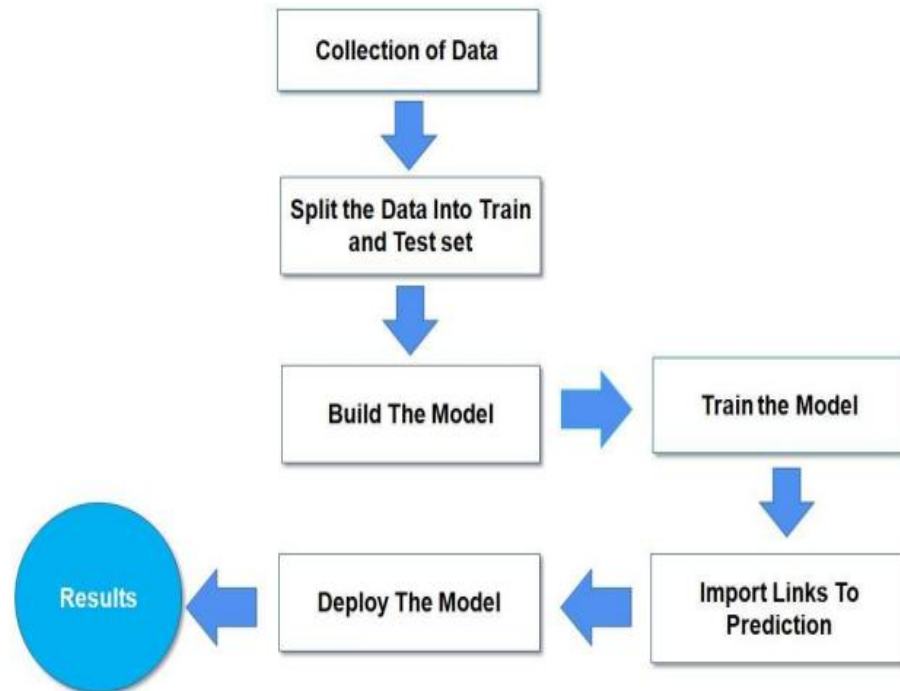


Fig. 6.1: System Architecture

The Architecture for Emotion Detection using machine learning can involve several steps, such as:

1.Data Collection

1. Dataset: A publicly available EEG dataset, containing brainwave activity from multiple subjects during emotional stimuli (e.g., happy, sad, neutral, etc.).
2. Classes: The dataset includes EEG data labeled into several emotional states:
 1. Happy
 2. Sad
 3. Neutral
 4. Angry

3. Data Format: EEG signals are stored in standard formats such as CSV or MAT, suitable for feature extraction and model input.

2. Data Preprocessing

1. Normalization: All EEG data is normalized to the range $[0, 1]$ to ensure consistency across different subjects and signals.
2. Augmentation:
 1. Spatial Transformations: Random shifting and scaling of signal windows to simulate real-world noise and variability.
 2. Temporal Augmentation: Random time-shifting of the EEG signals to ensure robustness during training.
3. Feature Extraction:
 1. Statistical Features: Features such as mean, standard deviation, skewness, and kurtosis are extracted from each signal.
 2. Time-Frequency Features: Spectral power and wavelet-based features are computed from the frequency bands of the EEG signals (e.g., alpha, beta, delta).
 3. GLCM Texture Features: 80 texture features are computed from the power spectral density maps of the EEG signal.
4. Train-Test Split: A stratified 80-20 split is applied to ensure that class distribution is preserved in both the training and evaluation sets.

3. Model Training

1. Machine Learning Models:
 1. Support Vector Machine (SVM): Used with an RBF kernel for robust separation of different emotional states.

2. Random Forest (RF): Ensemble of decision trees for high-dimensional feature classification.
3. LightGBM & XGBoost: Efficient gradient boosting methods for accurate classification and handling complex features.
4. Multi-Layer Perceptron (MLP): Deep neural network for nonlinear feature learning from the EEG signals.
5. Proposed Stacking Ensemble: Combines outputs of multiple top-performing models using a meta-classifier like Logistic Regression.

2. Training Configuration:

1. Optimizer: Adam or other built-in optimizers depending on the model.
2. Loss: Categorical Cross-Entropy or Binary Cross-Entropy for multi-class or binary classification.
3. Cross-validation: 5-fold cross-validation for ensuring model generalization and robustness.
4. Frameworks: scikit-learn, XGBoost, LightGBM, TensorFlow/Keras

4. Model Evaluation

1. Evaluation Metrics:

1. Accuracy, Precision, Recall, F1-score
2. Confusion Matrix: To analyze class-wise performance and identify misclassifications.

5. Deployment

1. Model Export: Trained models are saved in .pkl (for ML models) or .h5/.pt (for deep learning models) formats for deployment.

2. Serving: The models are integrated into a Flask-based web application for real-time emotion classification from EEG data.

6.2 UML DIAGRAMS

UML (Unified Modeling Language) diagrams serve as a standardized method to visualize and document the design of a software system. They are widely used by software engineers and system architects to represent the architecture, design decisions, code structure, workflows, and interaction patterns of complex systems. UML diagrams simplify intricate programming logic into comprehensible visual models, making it easier to analyze, build, and maintain software systems.

Importance of UML Diagrams in a Emotion Detection Project

In an **EEG-Based Emotion Detection System**, the project involves processing brain signal data, applying machine learning and deep learning models, and integrating the prediction system into a user interface. The system contains several components that work together to detect and classify human emotions from EEG signals. The process involves multiple components like:

1. Data collection modules
2. Preprocessing pipelines
3. Model training and evaluation
4. User interface for input
5. Prediction and output display
6. Feedback or monitoring system

UML diagrams help to simplify these relationships and present them visually, enabling a clearer understanding of how each part of the system fits into the bigger picture. This is especially useful not just for developers, but also for stakeholders, testers, or collaborators who may not be familiar with the underlying code but need to understand the system workflow.

In this project six basic UML diagrams have been explained:

1. Use Case Diagram
2. Class Diagram
3. Sequence Diagram
4. Activity Diagram
5. Component Diagram
6. Deployment Diagram

6.2.1 Class Diagram

A Class Diagram is a type of UML diagram that represents the static structure of a system by showing its classes, attributes, methods, and the relationships between them, helping developers understand how different components are organized and interact. In an EEG-Based Emotion Detection System, the class diagram typically includes classes such as User, EEG Data, Preprocessing, Feature Extraction, Feature Selection, Emotion Prediction, and Result Display, where each class performs a specific function: the User class manages user information, EEG Data handles the collection and storage of EEG signals, Preprocessing cleans and filters raw data to remove noise, Feature Extraction derives meaningful features from the signals, Feature Selection identifies the most relevant features, Emotion Prediction uses machine learning techniques to classify emotions, and Result Display presents the predicted outcomes to the user. Overall, the class diagram provides a clear blueprint of the system architecture, making the design, development, and maintenance process more efficient and systematic.

A Class Diagram in UML provides a detailed representation of the system's structure by outlining the classes, their properties, operations, and the relationships among them, offering a clear blueprint for system development. In the context of an EEG-Based Emotion Detection System, the diagram organizes core components such as User, EEGData, Preprocessing, Feature Extraction, Feature Selection, Emotion Prediction, and Result Display into well-defined modules, each contributing to a specific stage of the workflow.

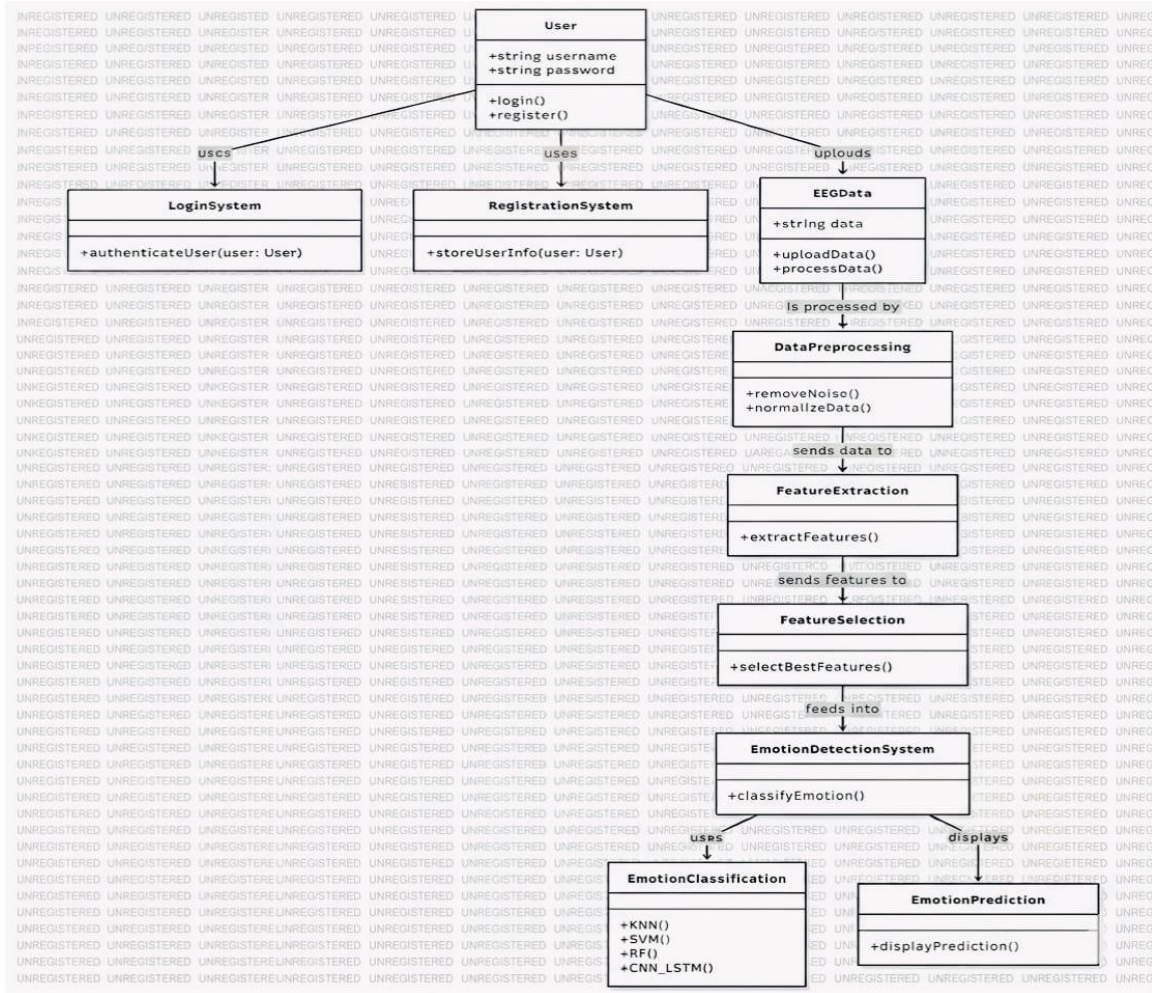


Fig. 6.2.1: Class Diagram

6.2.2 Use Case Diagram

A Use Case Diagram is a type of UML diagram that represents the interaction between users (actors) and the system. It shows what actions a user can perform in the system and how the system responds to those actions. In an EEG-Based Emotion Detection System, the use case diagram helps to illustrate the different functionalities available to users and how they interact with the emotion detection application.

The main actor in this system is the User, who interacts with the system through the interface. The user can perform several operations such as registering an account, logging into the system, uploading EEG data or selecting dataset input, and requesting emotion prediction. Once the data is provided, the system processes it through different modules

such as data preprocessing, feature extraction, feature selection using PSO, and machine learning or deep learning models.

After processing the EEG data, the system analyzes the patterns and predicts the emotional state of the user. The predicted emotion is then displayed on the user interface as the final output. The system may also allow users to view prediction results or history for analysis.

The Use Case Diagram helps in clearly defining the system functionalities and user interactions, making it easier to understand the workflow of the project. It is especially useful during the system design phase, as it provides a simple visual representation of how users interact with the emotion detection system

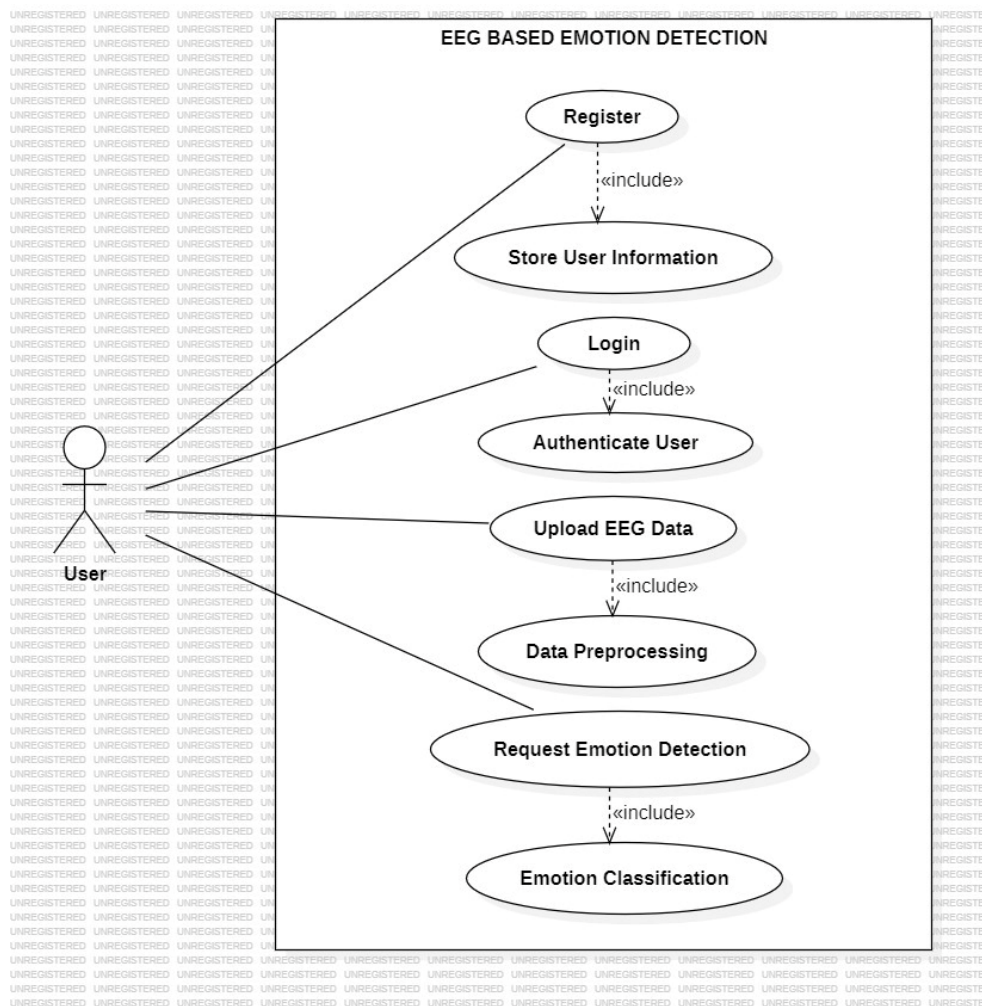


Fig. 6.2.2: Use Case Diagram

6.2.3 Sequence Diagram

A sequence diagram is a type of interaction diagram because it describes how and in what order a group of objects works together. These diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. Sequence diagrams are sometimes known as events.

UML sequence diagrams — sometimes known as event diagrams — show the order in which your objects interact. This includes the lifelines of your objects, the processes that interact with your objects, and the messages exchanged between the objects to perform a function. Developers and business professionals often use these diagrams to understand how to structure a new system or improve an existing process.

A sequence diagram simply depicts the interaction between the objects in a sequential order i.e. the order in which these interactions occur. We can also use the terms event diagrams or event scenarios to refer to a sequence diagram.

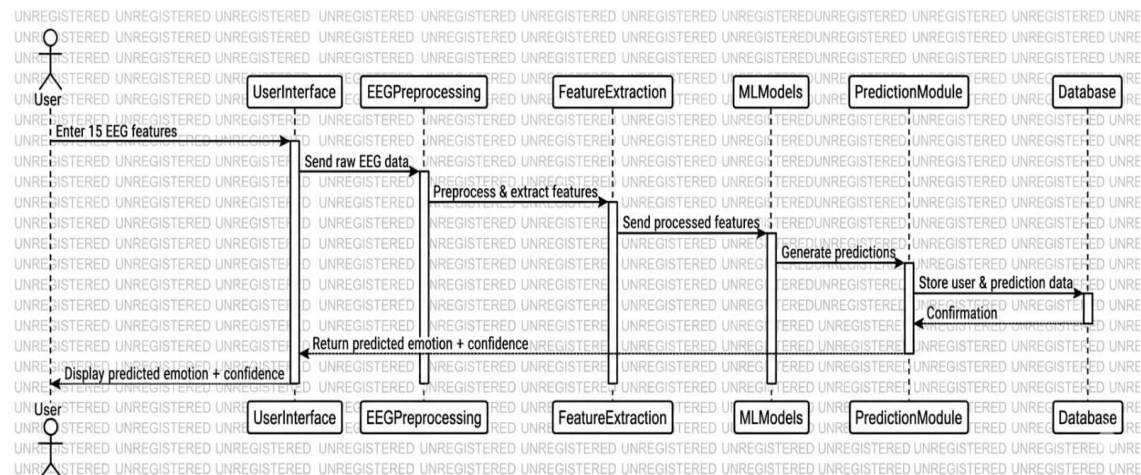


Fig. 6.2.3: Sequence Diagram

6.2.4 Activity Diagram

Activity diagrams are graphical representations of workflows that illustrate step-by-step activities and actions, including support for decision-making, iteration, and concurrency. In the Unified Modeling Language (UML), they are used to model the business and operational workflows of system components in a clear and structured

manner. An activity diagram shows the overall flow of control from one activity to another, helping to visualize the sequence of processes.

Additionally, activity diagrams improve understanding of complex systems by simplifying the representation of processes and interactions. They are also useful for identifying inefficiencies, optimizing workflows, and enhancing system design and documentation.

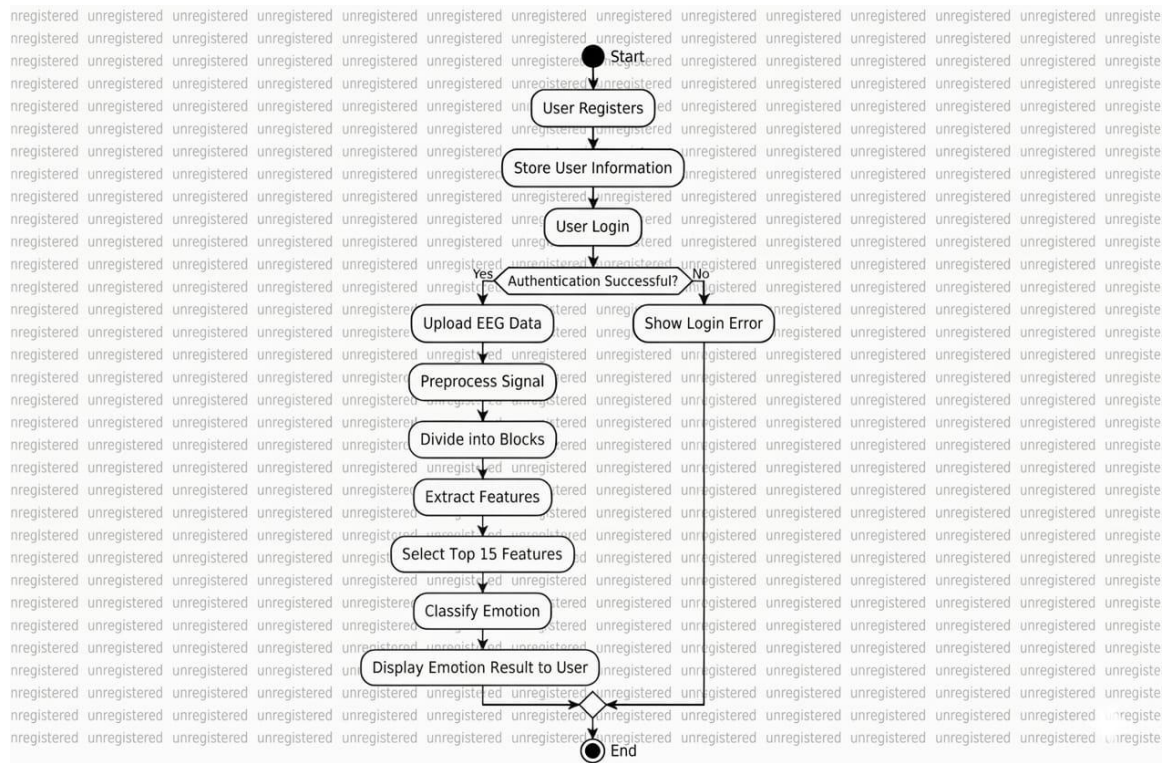


Fig. 6.2.4: Activity Diagram

6.2.5 Component Diagram

A component diagram is a type of UML structural diagram used to model the physical aspects of a system. It breaks down a complex system into smaller, manageable components. Each component represents a modular part of the system, such as a software module, library, or executable, that can be independently developed and later integrated into the complete system. By visualizing components and their relationships, developers

can better understand the system's structure and dependencies.

In the context of the EEG-Based Emotion Detection System, the component diagram shows key modules such as:

1. User Interface: Handles input of EEG features and displays emotion predictions.
2. Database Module: Stores user information and past prediction logs.
3. Data Preprocessing Module: Processes raw EEG inputs and prepares them for the model.
4. Feature Extraction Module: Extracts relevant features from EEG signals (15 features).
5. Machine Learning Layer: Includes CNN-LSTM, DeepConvNet, and other models used for emotion classification.
6. Prediction Module: Executes the trained model, calculates probabilities, and outputs predicted emotions.

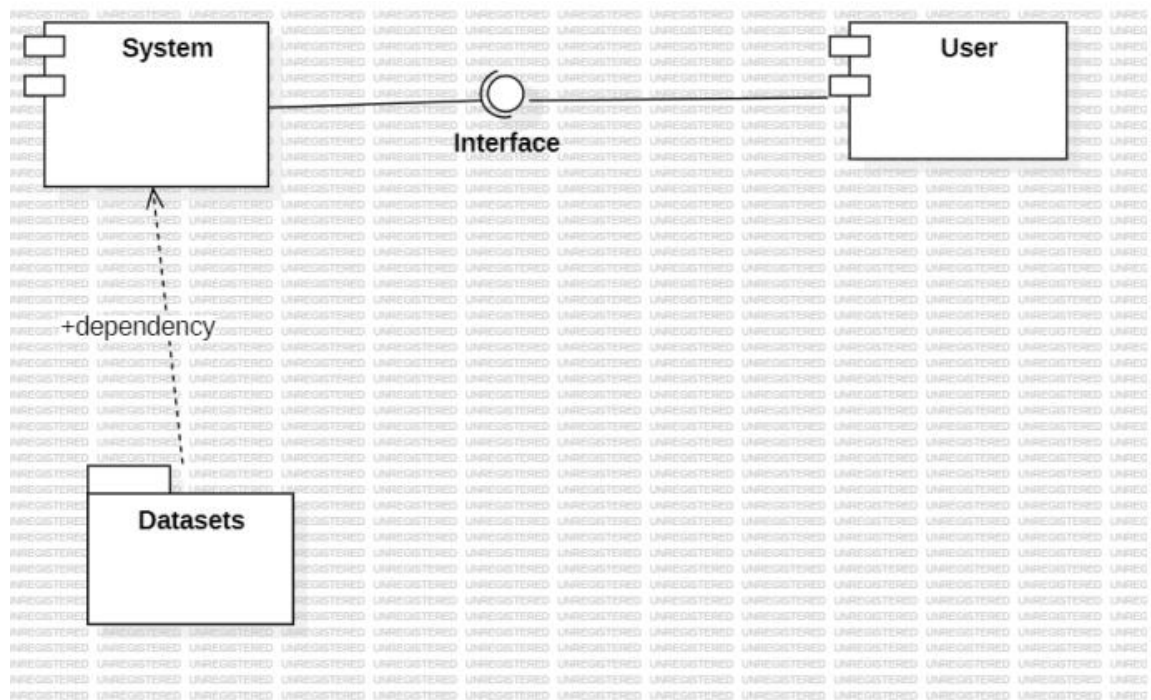


Fig. 6.2.5: Component Diagram

6.2.6 Deployment Diagram

A Deployment Diagram is a vital part of the Unified Modeling Language (UML) used to model the physical aspects of a software system. It represents the deployment

view, which focuses on how and where the software components—such as executables, libraries, databases, and web services—are physically deployed across various hardware nodes in a networked environment. While component diagrams show the logical relationships between software modules, the deployment diagram ties these modules to actual hardware devices, thereby providing a complete picture of the system's infrastructure.

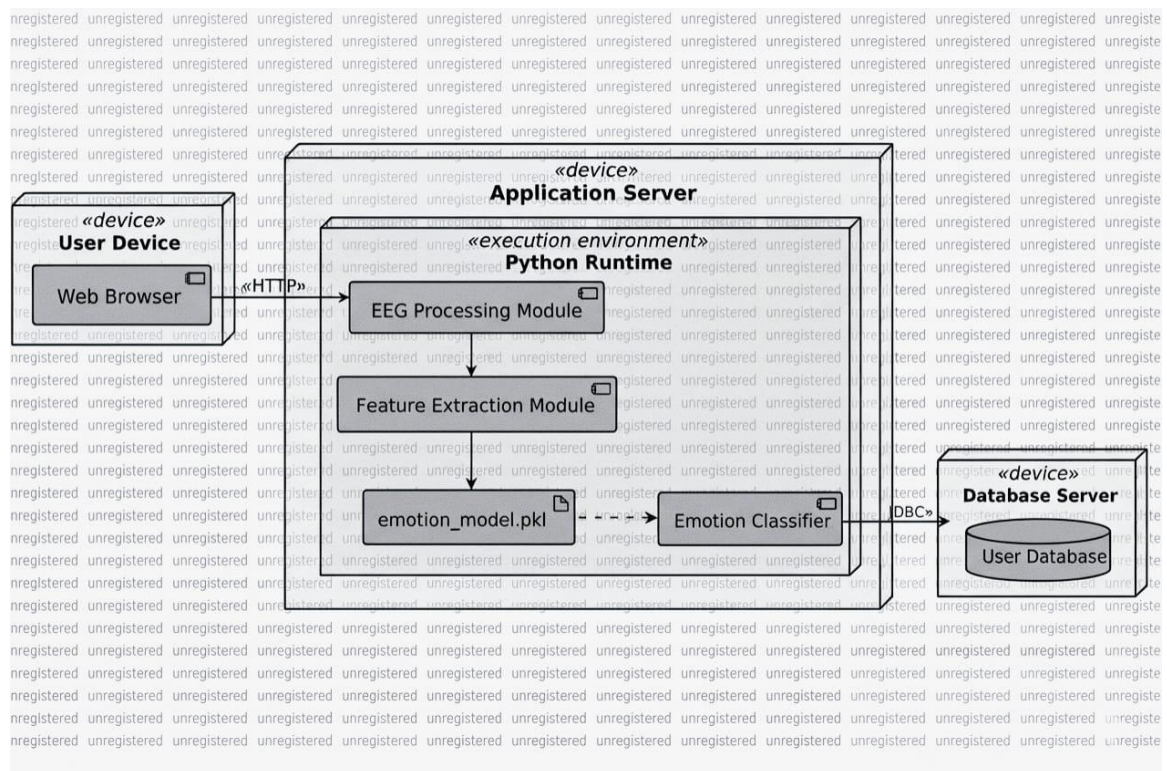


Fig. 6.2.6: Deployment Diagram

Chapter 7

SYSTEM IMPLEMENTATION

7.1 ALGORITHM

The EEG-based emotion detection system starts with collecting raw EEG signals from multiple subjects under different emotional stimuli. The data is pre-processed by filtering noise, normalizing values, and segmenting signals for analysis. Feature extraction is performed using the Extra Trees Classifier to select the most relevant EEG features, which are then optimized using the PSO Feature Model Selector. The selected features are input to machine learning classifiers like KNN, SVM, and Random Forest, and deep learning models such as CNN-LSTM and DeepConvNet are applied to capture spatial and temporal dependencies. Additionally, Roberta Similarity is used to evaluate EEG pattern similarity, improving classification accuracy. The system outputs predicted emotional states, which are validated using metrics such as accuracy, precision, recall, and F1-score, ensuring robust and real-time emotion detection.

7.1.1 EXTRA TREE CLASSIFIER:

The Extra Trees Classifier (ETC) is an ensemble learning method used for feature extraction in EEG-based emotion detection. It constructs multiple randomized decision trees by selecting random subsets of features and choosing optimal splits at each node, which reduces overfitting and improves model stability. ETC evaluates the importance of each feature based on its contribution to decision-making across all trees, allowing irrelevant or redundant features to be discarded. By retaining only the most significant features, ETC reduces dimensionality, captures complex non-linear relationships in EEG data, and provides robust inputs for further processing by classifiers such as KNN, SVM, and Random Forest. Its parallelizable nature also ensures faster computation, making it highly effective for handling high-dimensional and noisy EEG signals.

7.1.2 PSO Feature Model Selector for Feature Selection

The second model in the proposed system is the PSO Feature Model Selector, which utilizes Particle Swarm Optimization (PSO) to optimize the feature selection process.

This model builds on the features extracted by the Extra Trees Classifier and refines the selection to improve emotion detection accuracy.

Methodology:

1. **PSO Algorithm:** PSO is a bio-inspired optimization algorithm that simulates the movement of particles in a search space to find optimal solutions. Each particle represents a potential feature subset, and particles "fly" through the feature space, adjusting their position based on their own experiences and the experiences of neighboring particles.
2. **Feature Selection :** The PSO Feature Model Selector evaluates different features by training classifiers like KNN, SVM, and RF on these subsets. The particles explore different feature subsets, and the best-performing subset (based on accuracy) is selected for the next stages of the model.
3. **Optimization:** PSO iteratively refines the feature set, converging towards the most informative features that enhance the performance of subsequent classifiers.

By optimizing feature subsets, the PSOFeatureModelSelector reduces dimensionality and improves classification accuracy.

7.1.3 CNN-LSTM Hybrid Model

The CNN-LSTM hybrid model is a deep learning architecture designed to capture both spatial and temporal patterns in EEG signals for emotion detection. The **Convolutional Neural Network (CNN)** layers first extract spatial features by identifying local patterns and correlations across EEG channels, such as frequency bands or transient brainwave activities. These features are then passed to the **Long Short-Term Memory (LSTM)** layers, which model the temporal dependencies and sequential nature of EEG data, capturing how brain activity evolves over time. By combining CNN and LSTM, the model leverages the strengths of both architectures—CNN for robust spatial feature extraction and LSTM for learning temporal relationships—resulting in improved classification accuracy and the ability to detect complex emotional states in real-time.

7.1.4 Roberta Similarity for Signal Pattern Evaluation

The fourth model in the proposed system is Roberta Similarity, which is utilized to enhance emotion classification by evaluating the similarity between EEG signal patterns. This approach uses Roberta (a transformer-based model) for measuring the closeness of feature representations in the high-dimensional EEG data.

Methodology:

1. **Signal Pattern Comparison:** Roberta Similarity compares EEG signal patterns by analyzing the embedded feature representations. The model is trained to understand the relationships between various patterns in the EEG data that correspond to different emotional states.
2. **Pattern Matching:** Roberta uses its pre-trained transformer architecture to match EEG signal patterns, improving the model's ability to classify emotions accurately by detecting similar patterns in the data. This step is particularly useful for enhancing classification when dealing with subtle variations in EEG signals that might not be captured through traditional feature extraction alone.
3. **Similarity Scoring:** The model computes similarity scores between the input EEG signals and known emotional states, improving the accuracy of predictions. Higher similarity scores correspond to a more accurate classification of emotions.

By using Roberta Similarity, the model enhances the emotion detection system's ability to recognize nuanced patterns, improving overall classification performance. This step ensures that the model makes more precise predictions, even for ambiguous or overlapping emotional states.

7.1.5 DeepConvNet for Hierarchical Feature Extraction

The fifth model in the proposed system is **DeepConvNet**, a deep convolutional network specifically designed for feature extraction from EEG signals. DeepConvNet is highly effective in automatically learning hierarchical features from raw EEG data, which is crucial for improving the accuracy of emotion classification.

Methodology:

1. **Convolutional Layers for Feature Extraction:** DeepConvNet uses several convolutional layers to learn spatial features from raw EEG signals. These layers apply filters that capture local patterns in the EEG data, such as specific frequency bands or transient brainwave activities related to emotional responses. These features are crucial for detecting emotions but are often difficult to manually extract.
2. **Hierarchical Learning:** Unlike traditional feature extraction methods, DeepConvNet learns features at multiple levels of abstraction. The deeper layers of the network capture increasingly complex and abstract representations of the EEG signals, enabling the model to understand intricate patterns that may correspond to specific emotional states.
3. **End-to-End Learning:** DeepConvNet is trained end-to-end, meaning that it automatically learns both the relevant features and the mapping between these features and emotional states without the need for hand-crafted feature engineering. This ability allows DeepConvNet to adapt to the complexity of EEG data more efficiently than traditional methods.
4. **Improved Performance:** By leveraging DeepConvNet for hierarchical feature extraction, the model enhances the emotion detection system's ability to identify and classify emotions with high accuracy. This model also improves the system's generalization capabilities by capturing complex patterns across different emotional states and individuals.

In summary, DeepConvNet plays a critical role in the emotion detection system by learning effective representations directly from raw EEG signals without the need for manual feature extraction. It automatically captures both spatial and temporal patterns present in brain activity, enabling the system to better understand complex emotional states. This results in improved accuracy, reliability, and robustness of emotion classification, making the system more effective in real-world applications.

7.2 WORKING AND MODULE EXPLANATION

EEG-based emotion detection using Roberts Similarity and PSO Feature Selection is a process of analyzing brain signals to determine a person's emotional state. The prediction is based on the analysis of EEG signals recorded from multiple electrodes on the scalp. Machine learning and deep learning algorithms are used to extract, select, and classify relevant features from the EEG signals to predict emotions as Positive, Negative, or Neutral.

The process of emotion detection involves several steps. The first step is data collection, where EEG data is obtained from public datasets containing brainwave activity from multiple subjects during emotional stimuli. The collected data is then pre-processed, which involves cleaning, filtering, segmenting, and normalization to ensure it is suitable for analysis. Noise removal and scaling are essential for improving model performance.

Next, feature extraction is performed to capture meaningful characteristics from EEG signals. The Extra Trees Classifier is used to select the most relevant features, which include statistical, time-frequency, and texture-based features. Following this, feature selection is applied using the PSO Feature Model Selector, which optimizes the selection process using Particle Swarm Optimization and classifiers like KNN, SVM, and Random Forest. This ensures that only the most informative features are passed to the classification stage.

For emotion classification, hybrid deep learning models like CNN-LSTM are used to capture spatial and temporal dependencies in EEG data. Roberts Similarity is incorporated to quantify pattern similarity between EEG signals, enhancing classification accuracy. DeepConvNet is also employed to extract hierarchical features directly from raw EEG signals, improving the system's ability to recognize complex emotional states.

Finally, the system predicts the emotional state of the user as Positive, Negative, or Neutral, along with a confidence score indicating the reliability of the prediction. This output helps in real-time analysis and can be used in applications such as mental health monitoring, human-computer interaction, and affective computing.

7.2.1 MODULES INVOLVED

This project involves six main modules:

1. **Data Collection and Processing:** EEG datasets containing emotional responses from multiple subjects are collected from reliable sources. Raw EEG signals are pre-processed to remove noise, filter frequencies, segment signals, and normalize the data for analysis.
2. **Feature Extraction:** The Extra Trees Classifier is used to extract meaningful features from EEG signals, including statistical measures, frequency-based features, and GLCM texture features.
3. **Feature Selection:** The PSO Feature Model Selector optimizes feature subsets by evaluating their contribution to classification accuracy using KNN, SVM, and Random Forest models. Irrelevant or redundant features are removed.
4. **Model Training and Classification:** Hybrid deep learning models such as CNN-LSTM and DeepConvNet are trained using the selected features. Roberts Similarity is used to assess pattern similarity between EEG signals, improving emotion classification performance.
5. **Prediction System:** Users input EEG data, and the system predicts the emotional state as Positive, Negative, or Neutral, along with a confidence score indicating reliability.
6. **User Interface and Deployment:** A web-based interface displays the predicted emotion and confidence score to the user.

CHAPTER 8

SOURCE CODE

PYTHON CODE OF APP.PY:

```
from flask import Flask, render_template, request
```

```
import pymysql
```

```
import pandas as pd
```

```
import numpy as np
```

```
import tensorflow as tf
```

```
app = Flask(__name__)
```

Database connection setup

```
mydb = pymysql.connect(
```

```
    host="localhost",
```

```
    user="root",
```

```
    password="Deepthi@0905",
```

```
    port=3306,
```

```
    database='phishing'
```

```
)
```

```
mycursor = mydb.cursor()
```

Function to execute INSERT/UPDATE/DELETE queries

```
def executionquery(query, values):
```

```
    mycursor.execute(query, values)
```

```
    mydb.commit()
```

Function to retrieve data with parameterized query

```
def retrivequery1(query, values):
```

```
    mycursor.execute(query, values)
```

```
    data = mycursor.fetchall()
```

```
    return data
```

Function to retrieve data without parameters

```
def retrivequery2(query):
```

```
    mycursor.execute(query)
```

```
    data = mycursor.fetchall()
```

```
    return data
```

Define the improved_roberts_similarity function for the TensorFlow model

```
def improved_roberts_similarity(x):
```

```
    """Enhanced version for time series data"""
```

```
    vertical_diff = x[:, 1:, :] - x[:, :-1, :] # difference along time axis
```

```
    horizontal_diff = x[:, :, 1:] - x[:, :, :-1] # difference along feature axis
```

```
    horizontal_diff = tf.pad(horizontal_diff, [(0, 0), (0, 0), (0, 1)]) # pad to match shape
```

```
horizontal_diff = horizontal_diff[:, :-1, :]  
  
magnitude = tf.sqrt(tf.square(vertical_diff) + tf.square(horizontal_diff))  
  
magnitude = tf.math.l2_normalize(magnitude, axis=1) # normalize magnitude  
  
return magnitude  
  
# Load the TensorFlow EEG model  
  
try:  
  
    model = tf.keras.models.load_model(  
  
        'eeg_cnn_lstm_roberts.h5',  
  
        custom_objects={'improved_roberts_similarity': improved_roberts_similarity}  
  
    )  
  
except Exception as e:  
  
    print(f"Error loading TensorFlow model: {e}")  
  
# Route for main landing page  
  
@app.route('/')  
  
def index():  
  
    return render_template('index.html')  
  
# Route for home page  
  
@app.route('/home')  
  
def home():  
  
    return render_template('home.html')
```

Route for algorithm explanation page

```
@app.route('/algorithm')
```

```
def algorithm():
```

```
    return render_template('algorithm.html')
```

Route for user registration

```
@app.route('/register', methods=["GET", "POST"])
```

```
def register():
```

```
    if request.method == "POST":
```

```
        email = request.form['email']
```

```
        password = request.form['password']
```

```
        c_password = request.form['c_password']
```

```
        if password == c_password:
```

```
            query = "SELECT UPPER(email) FROM users"
```

```
            email_data = retrievequery2(query)
```

```
            email_data_list = [i[0] for i in email_data]
```

```
            if email.upper() not in email_data_list:
```

```
                query = "INSERT INTO users (email, password) VALUES (%s, %s)"
```

```
                values = (email, password)
```

```
                executionquery(query, values)
```

```
                return render_template('login.html', message="Successfully Registered!")
```



```
return render_template('register.html', message="This email ID already exists!")
```

```
return render_template('register.html', message="Passwords do not match!")
```

```
return render_template('register.html')
```

Route for user login

```
@app.route('/login', methods=["GET", "POST"])
```

```
def login():
```

```
    if request.method == "POST":
```

```
        email = request.form['email']
```

```
        password = request.form['password']
```

```
        query = "SELECT UPPER(email) FROM users"
```

```
        email_data = retrievequery2(query)
```

```
        email_data_list = [i[0] for i in email_data]
```

```
        if email.upper() in email_data_list:
```

```
            query = "SELECT password FROM users WHERE email = %s"
```

```
            values = (email,)
```

```
            password_data = retrievequery1(query, values)
```

```
            if password == password_data[0][0]:
```

```
                global user_email
```

```
                user_email = email # store logged-in user email
```

```
                return render_template('home.html')
```

```
return render_template('login.html', message="Invalid Password!")
```

```
return render_template('login.html', message="This email ID does not exist!")
```

```
return render_template('login.html')
```

Route for EEG-based prediction

```
@app.route('/prediction', methods=['GET', 'POST'])
```

```
def prediction():
```

```
    if request.method == "POST":
```

```
        try:
```

```
            # Define the order of input features
```

```
            feature_order = [
```

```
                'entropy3_b', 'entropy0_a', 'entropy0_b', 'entropy3_a', 'min_q_7_b',
```

```
                'mean_3_a', 'mean_3_b', 'mean_d_3_b2', 'max_q_8_b', 'min_q_15_a',
```

```
                'mean_d_15_a', 'min_0_a', 'fft_91_b', 'min_q_12_a', 'min_q_5_a'
```

```
            ]
```

Collect input features from form

```
            input_features = {}
```

```
            for feature in feature_order:
```

```
                try:
```

```
                    input_features[feature] = float(request.form[feature])
```

```
                except (KeyError, ValueError):
```

```
        return render_template('prediction.html', error_message=f'Invalid or missing  
value for {feature}')
```

Create DataFrame to maintain feature order

```
input_df = pd.DataFrame([input_features], columns=feature_order)
```

```
# Convert to numpy array and reshape to 3D for model
```

```
X_input = input_df.values
```

```
X_input_3d = X_input.reshape(X_input.shape[0], X_input.shape[1], 1)
```

Make prediction using TensorFlow model

```
y_pred_prob = model.predict(X_input_3d, verbose=0)
```

```
print("Raw prediction:", y_pred_prob)
```

```
predicted_class = np.argmax(y_pred_prob, axis=1)[0]
```

```
probabilities = y_pred_prob[0]
```

```
confidence = probabilities[predicted_class] * 100
```

Map predicted index to label

```
class_names = {0: "Negative", 1: "Neutral", 2: "Positive"}
```

```
predicted_label = class_names.get(predicted_class, "Unknown")
```

Format probabilities for display

```
prob_display = {class_names[i]: f"{prob:.4f}" for i, prob in  
enumerate(probabilities)}
```

```

        return render_template('prediction.html',

                                prediction=f'Prediction: {predicted_label} (Confidence:
{confidence:.2f}%)',
                                probabilities=prob_display)

    except Exception as e:

        print(f'Error during prediction: {e}')

        return render_template('prediction.html', error_message=f'An error occurred
during prediction: {str(e)}')

# GET request: show the prediction form

    return render_template('prediction.html')

# Run the Flask app

if __name__ == '__main__':

    app.run(debug=True)

```

CHAPTER 9

TESTING

9.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Function	Test Description	Input	Expected Output	Pass Criteria
Register User	Sign up	User Credentials	Redirect to Home Page	Credentials should match the data
Login Test Case 1	Trying with valid Credentials	User email & password	Redirect to Home Page	Credentials matching the data
Login Test Case 2	Invalid credentials	User email & password	Failed redirect	Incorrect credentials detected
EEG Prediction	Empty input	No EEG features	Error message	Should prompt for input
EEG Prediction	Valid		Predicted emotion	Correct

	features	EEG feature values	+ confidence	prediction displayed
--	----------	-----------------------	--------------	-------------------------

Table 9.1: Unit Testing

This process helps in identifying bugs early, ensuring that each internal logic—like feature extraction, model evaluation, and prediction—is producing correct outputs for given inputs. Since our system involves decision-making algorithms like the Random Forest Classifier, unit testing is critical to validate different decision paths and logic flows. Before integrating modules like prediction and drug recommendation into the full system, each unit is tested separately to ensure reliability and correctness, which ultimately contributes to the system's overall stability and performance.

9.2 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error. Additionally, it ensures that data is correctly transferred between integrated modules without loss or corruption. It also verifies that all interconnected components function together smoothly under different conditions.

Test Description	Input	Expected Output	Pass Criteria
EEG Data → Preprocessing → Model Prediction	Input EEG features	Predicted Emotion + Confidence	Accurate prediction displayed for all inputs.
Feature Extraction → Model Input → Prediction	EEG features reshaped & normalized	Model receives correct input shape	Prediction executed without errors
UI Interaction → Model Output Display	Submit prediction form	Prediction page shows label, confidence, probabilities	Display correct and responsive

Table 9.2: Integration Testing

This phase of testing helps uncover any mismatches, communication issues, or unexpected behaviors that might occur when components such as the model output and UI input/output handling are linked together. It ensures that even though individual components work fine in isolation, they also perform well together, delivering a smooth and functional experience for the end user.

9.3 Functional testing

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

9.4 SYSTEM TESTING

The purpose of testing is to discover errors and ensure that the software system performs as expected under various conditions. Testing involves identifying faults or weaknesses in a work product and verifying the functionality of components, sub-assemblies, assemblies, and the final system. It ensures that the software meets specified requirements and user expectations without failing in an unacceptable manner. There are various types of testing, each designed to address specific aspects such as functionality, performance, security, and usability.

Testers do not require in-depth programming knowledge at this stage, as the focus is mainly on validating user interactions, checking output accuracy, and observing system responses. System testing also evaluates interface behavior, control flow, processing speed, and how effectively the system handles different types of data inputs. It helps in identifying hidden defects that may not have been detected during unit or integration testing. Additionally, it ensures that all modules work together seamlessly in a real-world environment.

Furthermore, testing improves the overall quality, reliability, and performance of the software product. It provides confidence that the system is stable, secure, and ready for deployment to end-users. By thoroughly examining the system, testing minimizes risks, reduces maintenance costs, and enhances user satisfaction by delivering a smooth and error-free experience.

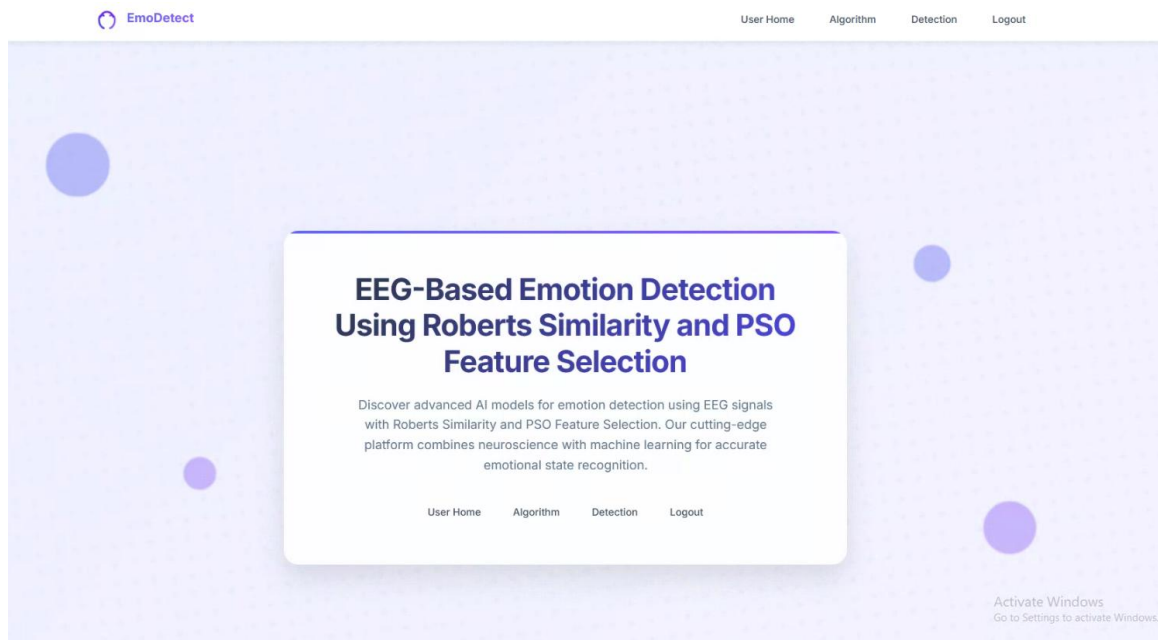


Fig. 8.4: System Testing

9.5 ACCEPTANCE TESTING

Acceptance Testing is the final level of software testing that validates whether the developed system meets business requirements and is ready for real-time use. In this project, acceptance testing ensures that the EEG-Based Emotion Detection System functions correctly from the user's perspective. It verifies overall functionality, ease of use, prediction accuracy, and that predicted emotions are displayed accurately along with confidence percentages.

This testing phase confirms that the complete system behaves as expected in real-world scenarios. It ensures that users can interact with the interface, enter the required EEG features, receive meaningful emotion predictions, and view associated probabilities.

EEG Features Used for Prediction:

1. entropy3_b, entropy0_a, entropy0_b, entropy3_a, min_q_7_b,

2. mean_3_a, mean_3_b, mean_d_3_b2, max_q_8_b, min_q_15_a,
3. mean_d_15_a, min_0_a, fft_91_b, min_q_12_a, min_q_5_a

Sample Test Cases for Acceptance Testing:

1. When valid values for all 15 EEG features are entered, the system should accurately predict the corresponding emotion (Negative, Neutral, Positive).
2. Upon prediction, the system should display the predicted emotion along with confidence percentage and probabilities for all three classes.
3. If any of the 15 EEG features are missing or contain invalid values, the system should show an error message prompting correct input.
4. The system should not crash for invalid or non-numeric inputs and must provide a user-friendly error message.
5. When accessed from different screen sizes or devices, the interface should remain functional and responsive.
6. Completing the full workflow—from entering EEG features to viewing predictions—should execute smoothly without delays or interruptions.

9.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Test objectives

- 1.All field entries must work properly.

2. Pages must be activated from the identified link.
3. The entry screen, messages and responses must not be delayed.

Features to be tested

1. Verify that the entries are of the correct format
2. No duplicate entries should be allowed
3. All links should take the user to the correct page.

9.7 White Box Testing

White Box Testing verifies the internal logic of the EEG-Based Emotion Detection System. It checks that all 15 EEG input features are correctly processed, reshaped, and fed into the model, that the `improved_roberts_similarity` function works properly, and that the prediction logic correctly calculates probabilities and maps them to emotions.

Key Areas:

1. All EEG features (`entropy3_b`, `entropy0_a`, ..., `min_q_5_a`) are correctly read and converted.
2. Input reshaping and normalization are accurate for the model.
3. All decision branches and error handling in `prediction()` are tested.
4. Model outputs are correctly interpreted as Negative, Neutral, or Positive.

Chapter 10

ALGORITHM PERFORMANCE RESULTS

PSO Feature Model Selector's:

The PSO Feature Model Selector's classification output indicates a highly effective multi-class model (classes 0, 1, 2) on a 427-sample dataset. Overall accuracy is 97%, with macro and weighted averages of 0.97 across precision, recall, and F1-score, showing balanced performance. Class 1 excels (F1=0.99, near-perfect), while class 0 (F1=0.96) and class 2 (F1=0.95) are strong but exhibit minor errors. The confusion matrix reveals 133 true negatives (class 0 correctly predicted), 142 true neutrals (class 1), and 138 true positives (class 2), with limited misclassifications: 9 false positives from class 0 to 2, 3 false negatives from class 2 to 0, and 1 each from class 1 to 2 and class 2 to 1. This suggests robust feature selection via PSO, ideal for reliable sentiment or categorical prediction.

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.94	0.96	142
1	0.99	0.99	0.99	143
2	0.93	0.97	0.95	142
accuracy			0.97	427
macro avg	0.97	0.97	0.97	427
weighted avg	0.97	0.97	0.97	427

Fig 10.1 Classification Report

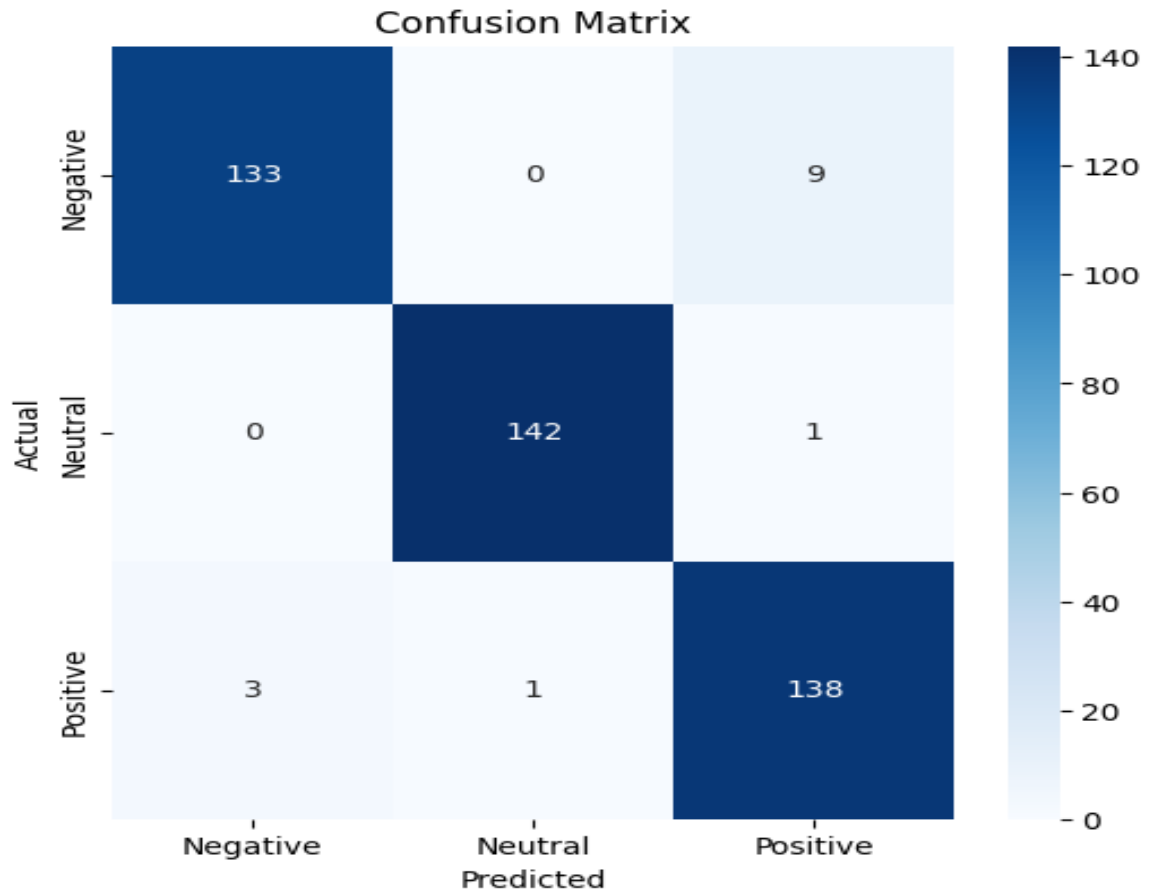


Fig 10.2 Confusion Matrix

CNN – LSTM with Roberta similarity:

The CNN-LSTM model with RoBERTa similarity embedding achieves solid performance on this 427-sample multi-class classification task (classes 0/negative, 1/neutral, 2/positive), with 90% overall accuracy. Macro and weighted averages are 0.92/0.90/0.89 for precision/recall/F1, indicating balanced but imperfect results. Class 1 shines (F1=0.99, near-perfect), while class 0 is strong in recall (1.00, F1=0.87) but lower in precision. Class 2 struggles most (F1=0.82, recall=0.70). The confusion matrix shows excellent negative/neutral detection (142 true negatives, 142 true neutrals), but 41 false negatives (positives misclassified as negatives) and 2 false neutrals drag down positives, with only 1 neutral-to-positive error. This suggests the model excels at conservative

predictions but may need tuning for positive class sensitivity in sentiment analysis or similar tasks.

Classification Report:

	precision	recall	f1-score	support
0	0.78	1.00	0.87	142
1	0.99	0.99	0.99	143
2	0.99	0.70	0.82	142
accuracy			0.90	427
macro avg	0.92	0.90	0.89	427
weighted avg	0.92	0.90	0.89	427

Fig 10.3 Classification Report

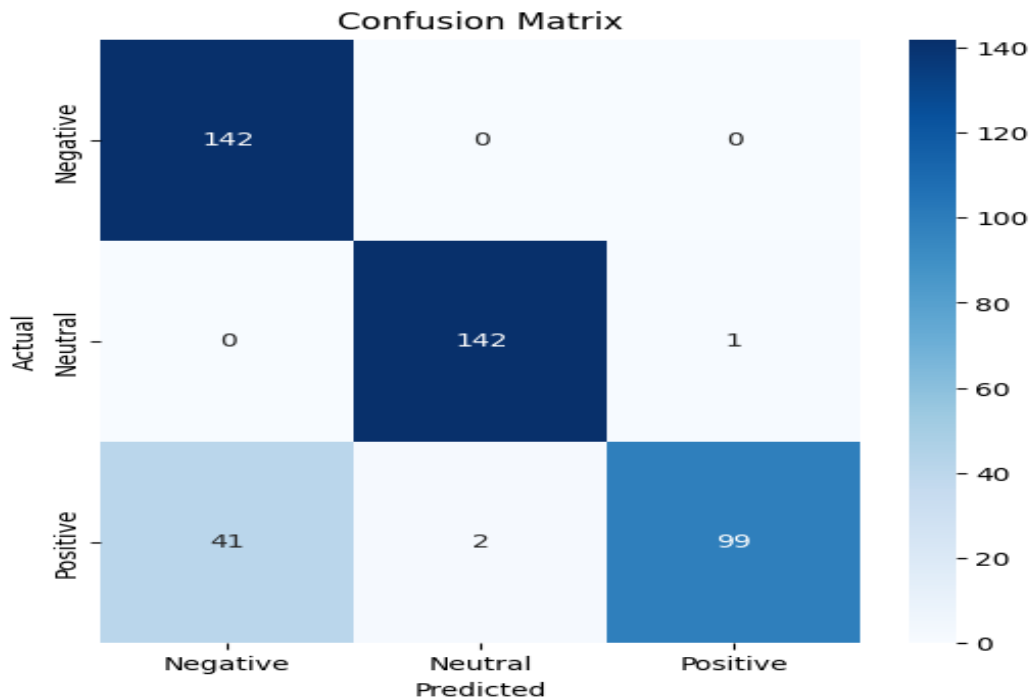


Fig 10.4 Confusion matrix

DeepConvNet:

The DeepConvNet model demonstrates strong performance on this 427-sample sentiment classification task (Negative, Neutral, Positive), achieving 95% accuracy. Macro and weighted averages are 0.95 across precision, recall, and F1-score, indicating consistent results. Neutral class excels (F1=0.98), while Negative (F1=0.94) and Positive (F1=0.92) perform well but show slight weaknesses. The confusion matrix highlights 135 true negatives, 144 true neutrals, and 127 true positives, with minimal errors: 8 negatives misclassified as positives, 4 neutrals as positives, 1 neutral as negative, and 8 positives as negatives. This suggests effective feature extraction by DeepConvNet, suitable for sentiment analysis, though minor tuning could enhance positive class precision.

Additionally, the model's high accuracy and balanced performance across all classes indicate its robustness and reliability for practical applications. The low number of misclassifications suggests that DeepConvNet effectively captures subtle patterns in the data, leading to stable predictions. With further fine-tuning, such as optimizing hyperparameters or increasing training data, the model's performance—especially for the positive class—can be further improved, making it even more efficient for real-world sentiment analysis tasks.

Classification Report:				
	precision	recall	f1-score	support
Negative	0.94	0.94	0.94	143
Neutral	0.99	0.97	0.98	148
Positive	0.91	0.93	0.92	136
accuracy			0.95	427
macro avg	0.95	0.95	0.95	427
weighted avg	0.95	0.95	0.95	427

Fig 10.5 Classification Report

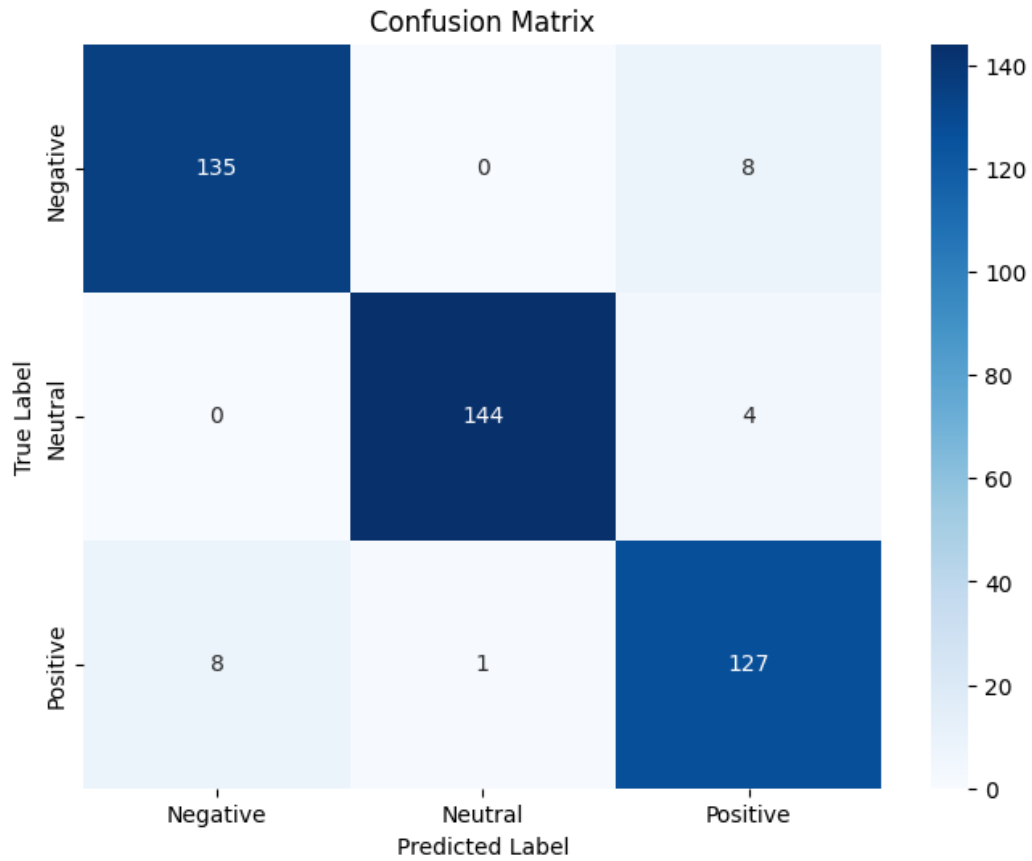


Fig 10.6 Confusion matrix

Chapter 11

SCREEN LAYOUTS

Landing Homepage:

This landing page is for an EEG-based emotion detection system that uses Roberts Similarity and PSO Feature Selection to analyze brain signals. The site's clean design highlights the platform's ability to combine neuroscience with machine learning for accurate emotion recognition.

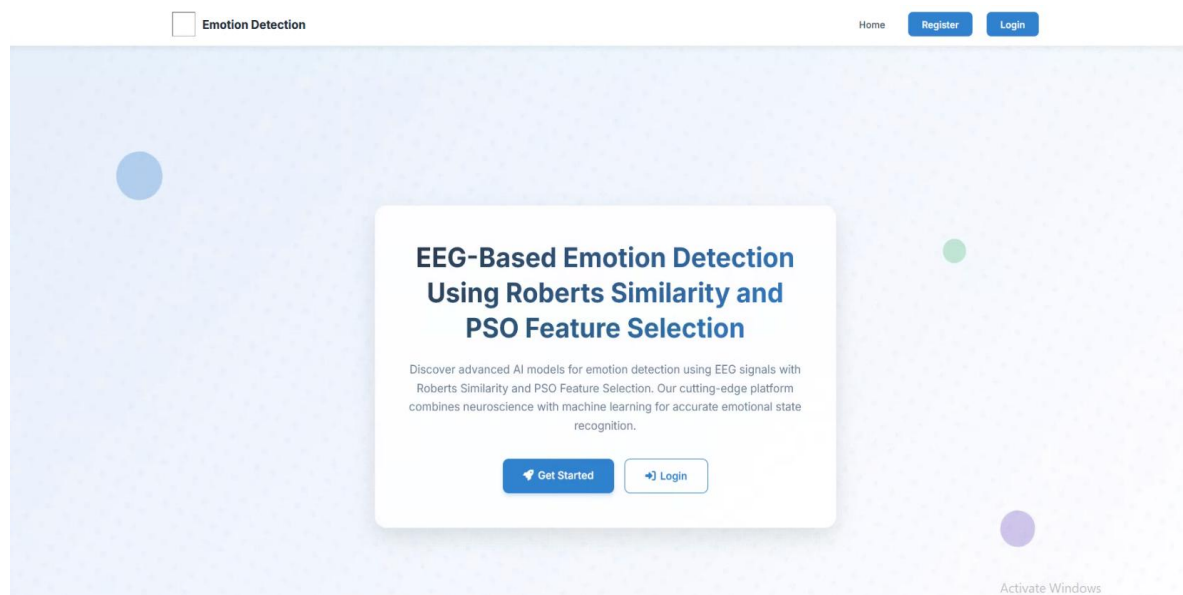
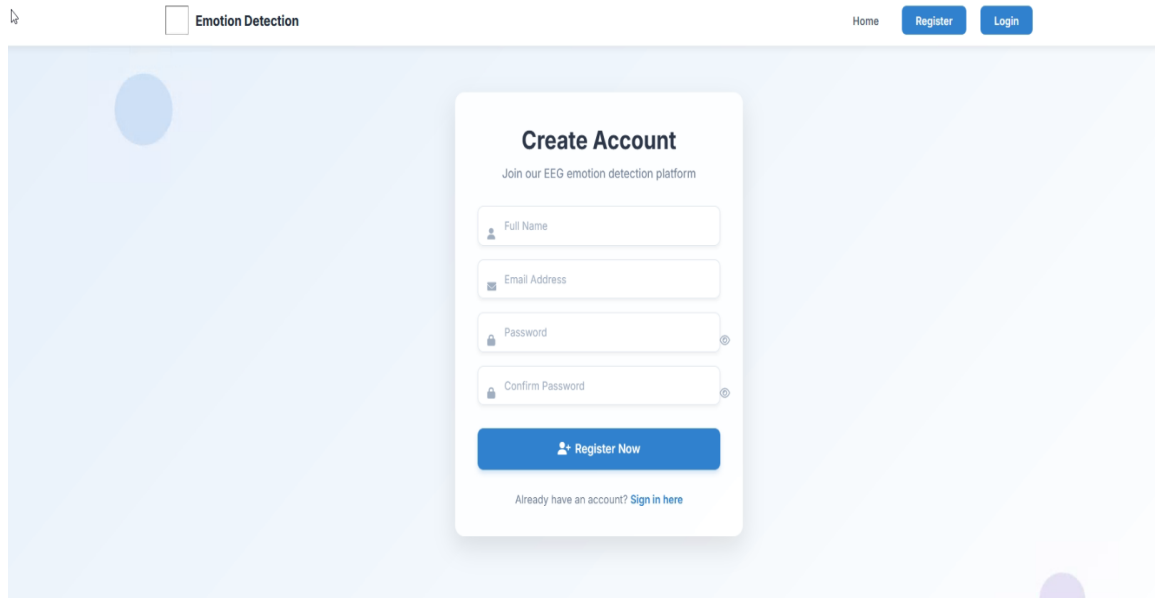


Fig. 11.1: Home Page

Register Page:

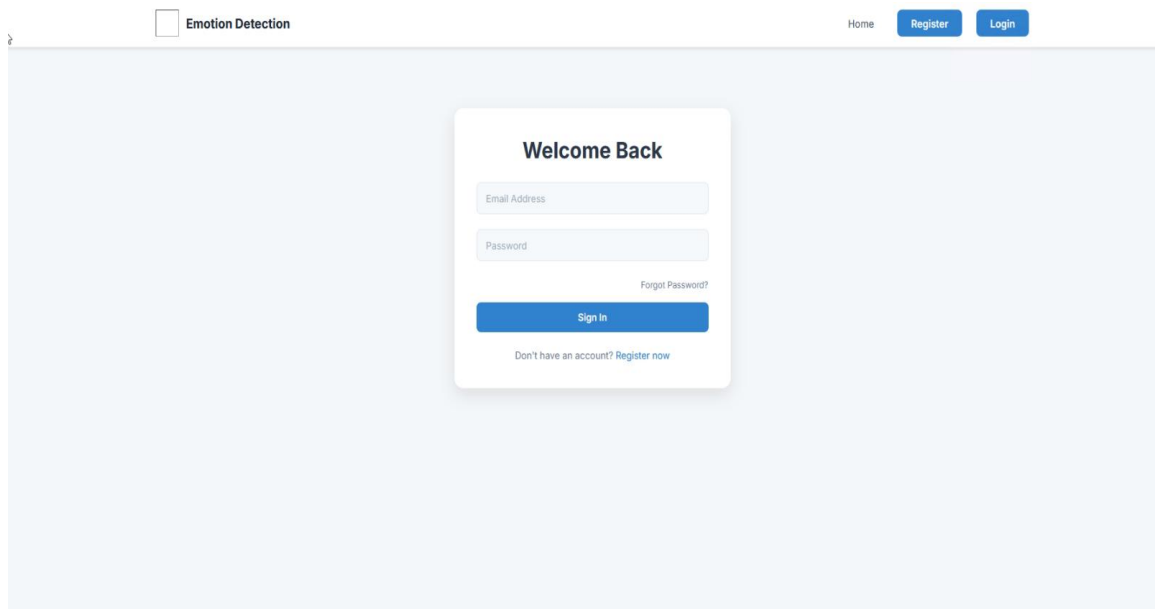
This page allows new users to sign up by entering personal details like name, age, email, and password. The form ensures secure and authenticated access to the prediction system. It also validates user inputs to prevent errors and maintain data integrity. Additionally, it helps create a personalized user profile for better interaction with the system.



The screenshot shows the 'Register' page of the 'Emotion Detection' application. The page has a light blue background with a white central form. The form is titled 'Create Account' and includes the subtext 'Join our EEG emotion detection platform'. It contains four input fields: 'Full Name', 'Email Address', 'Password', and 'Confirm Password'. Each field has a small icon on the left (person, envelope, and padlocks respectively) and a small circular icon on the right. Below the fields is a blue 'Register Now' button with a person icon. At the bottom of the form, there is a link: 'Already have an account? [Sign in here](#)'. The top navigation bar includes a 'Home' link and 'Register' and 'Login' buttons.

Fig. 11.2: Register Page

Login Page: Registered users can securely log into the system using their email and password. This page validates user credentials before granting access to the prediction features.



The screenshot shows the 'Login' page of the 'Emotion Detection' application. The page has a light blue background with a white central form. The form is titled 'Welcome Back'. It contains two input fields: 'Email Address' and 'Password'. Below the 'Password' field is a link: 'Forgot Password?'. Below the fields is a blue 'Sign In' button. At the bottom of the form, there is a link: 'Don't have an account? [Register now](#)'. The top navigation bar includes a 'Home' link and 'Register' and 'Login' buttons.

Fig. 11.3: Login Page

Dashboard:

This image shows a logged-in version of the EEG-based emotion detection platform, now branded as EmoDetect. The main content remains the same, highlighting the use of Roberts Similarity and PSO Feature Selection, but the navigation has changed to a user-focused menu with options like "User Home," "Algorithm," "Detection," and "Logout." This indicates that the user has successfully registered and is now inside the application.

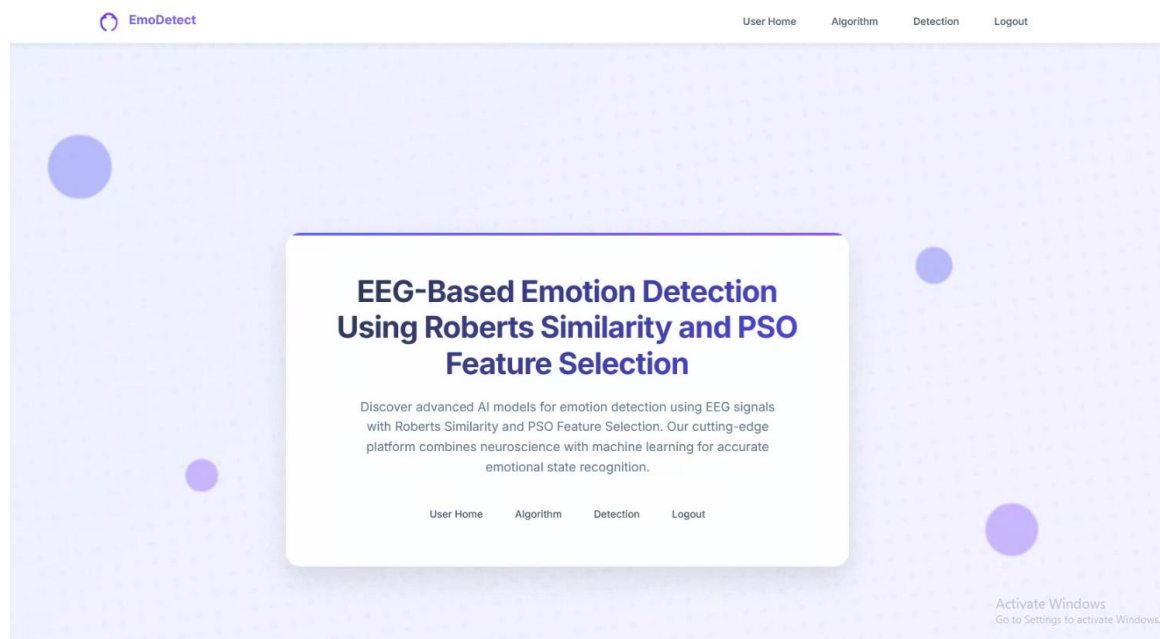


Fig. 11.4: Emotion-Detection Dashboard

This page showcases the performance evaluation of the EmoDetect system's emotion detection model. It presents a detailed classification report that breaks down the model's performance by emotion, showing key metrics like precision, recall, and F1-score for each category. It also helps users easily understand how well the model performs across different emotional classes. Additionally, it provides insights into areas where the model can be further improved for better accuracy and reliability.

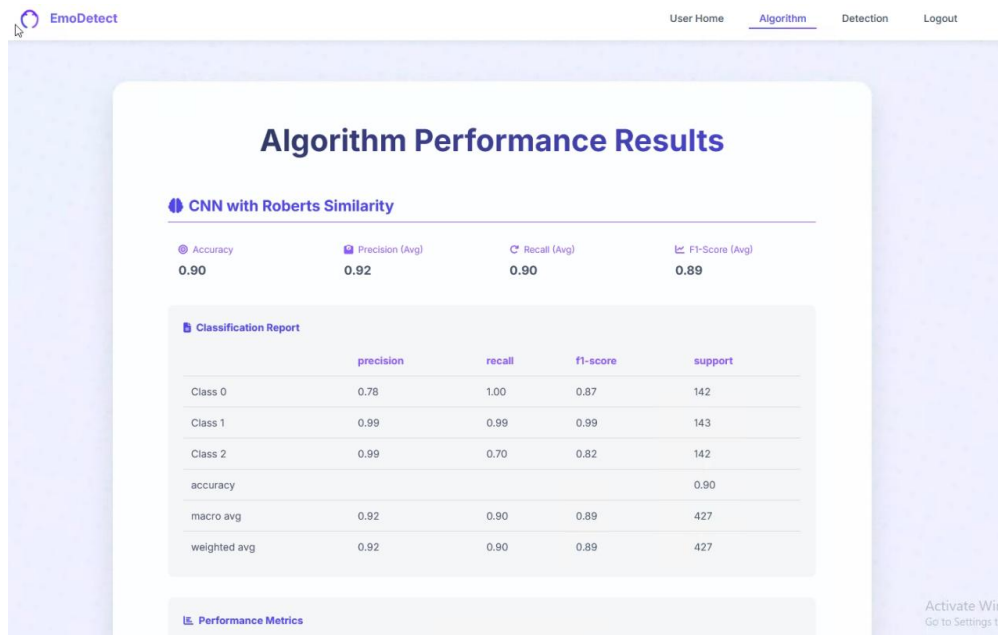


Fig. 11.5: Algorithm Performance Results-1

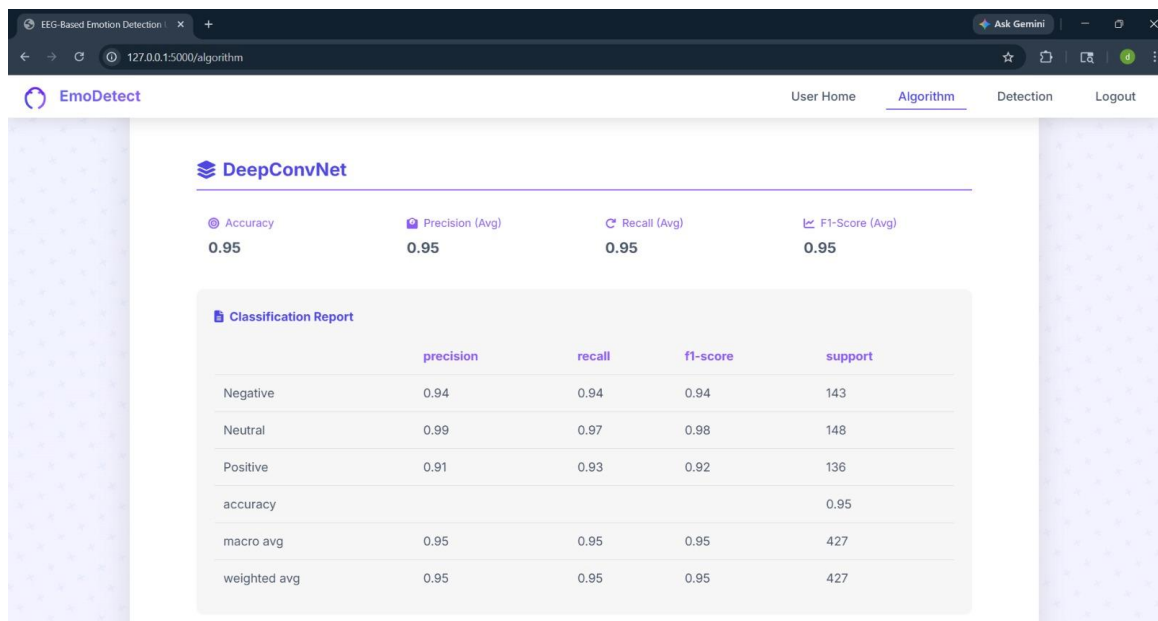


Fig. 11.6: Algorithm Performance Results-2

Prediction Page:

The Prediction page allows users to input the extracted 15 EEG features for analysis. These features are processed by the trained model to classify the emotion as Negative, Neutral, or Positive. The system analyzes the feature values and generates a prediction based on learned patterns. It displays the predicted emotion label along with confidence percentage and class-wise probabilities. This helps users clearly understand the model's decision and reliability of the prediction.

The screenshot displays the 'EmoDetect' web application interface. At the top, there's a navigation bar with 'User Home', 'Algorithm', 'Prediction' (active), and 'Logout'. The main content area is titled 'Emotion Detection'. A prominent purple box shows the prediction: 'Prediction: Positive (Confidence: 96.35%)'. Below this, there are sections for 'Entropy Features' and 'Statistical Features'. The 'Entropy Features' section contains four input fields: 'Entropy3_b' (placeholder: 'Enter entropy3_b value'), 'Entropy0_a' (placeholder: 'Enter entropy0_a value'), 'Entropy0_b' (placeholder: 'Enter entropy0_b value'), and 'Entropy3_a' (placeholder: 'Enter entropy3_a value'). A tooltip message 'Please fill out this field' is visible near the 'Entropy3_a' input. The 'Statistical Features' section is currently empty. The browser's address bar shows '127.0.0.1:5000/prediction'. The Windows taskbar at the bottom shows the date as 10-02-2026 and time as 13:33.

Fig. 11.7: Prediction Page-1

EmoDetect [User Home](#) [Algorithm](#) [Prediction](#) [Logout](#)

Max_q_8_b	Min_q_15_a
0.10	-1.00
Mean_d_15_a	Min_0_a
-0.50	-1.60
FFT Features	
Fft_91_b	Min_q_12_a
0.10	-1.00
Min_q_5_a	
-0.90	

Analyze The Values

Fig. 11.8: Prediction Page-2

Chapter 12

CONCLUSION

The comparative analysis of PSO Feature Model Selector, CNN-LSTM with RoBERTa similarity, and DeepConvNet models on a 427-sample classification task reveals distinct strengths and trade-offs. PSO Feature Model Selector achieves the highest accuracy (97%) with near-perfect F1-scores (0.96-0.99) across classes, supported by a confusion matrix showing minimal misclassifications (e.g., 9 false positives from class 0 to 2). This suggests superior feature selection and robustness for sentiment or categorical prediction. CNN-LSTM with RoBERTa similarity, at 90% accuracy, excels in neutral class prediction (F1=0.99) but struggles with positives (F1=0.82), with 41 false negatives indicating conservative bias. This model leverages deep contextual embedding effectively but may require adjustment for balanced sensitivity. DeepConvNet, with 95% accuracy, offers a balanced performance (F1=0.92-0.98), with a confusion matrix reflecting tight clustering around true labels (e.g., 144 true neutrals, 127 true positives) and limited errors (e.g., 8 negatives as positives). Its convolutional architecture likely enhances feature extraction, making it a strong contender for sentiment analysis.

Overall, PSO Feature Model Selector stands out for its precision and consistency, ideal for applications demanding high reliability. CNN-LSTM with RoBERTa suits tasks where neutral sentiment dominates but needs refinement for positive class detection. DeepConvNet provides a reliable middle ground, excelling in balanced prediction. The choice depends on specific needs: PSO for maximum accuracy, CNN-LSTM for contextual depth with tuning, and DeepConvNet for general robustness. Future work could explore hybrid approaches or hyperparameter optimization to address residual weaknesses, particularly in positive class misclassifications across all models.

Chapter 13

FUTURE ENHANCEMENTS

To enhance the performance of PSOFeatureModelSelector, CNN-LSTM with RoBERTa similarity, and DeepConvNet models, several strategies can be pursued. For PSO Feature Model Selector, integrating adaptive particle swarm optimization could refine feature selection, potentially boosting its already high 97% accuracy by minimizing rare misclassifications like the 9 false positives. CNN-LSTM with RoBERTa could benefit from fine-tuning RoBERTa embeddings with domain-specific data and adding attention mechanisms to improve positive class recall (currently 70%, with 41 false negatives), aiming to close the gap to its 90% accuracy ceiling. DeepConvNet could incorporate residual connections or batch normalization to enhance feature extraction, targeting its 95% accuracy by reducing errors like the 8 negatives misclassified as positives. Across all models, experimenting with data augmentation (e.g., synthetic samples) and ensemble techniques could address class imbalance and boost robustness. Additionally, leveraging real-time feedback loops to retrain on misclassified instances could adapt models to evolving datasets. Hardware optimization, such as GPU acceleration, and exploring transfer learning with larger pre-trained models could also accelerate training and improve generalization. These enhancements, tailored to each model's strengths, promise to push performance closer to perfection for sentiment analysis or similar tasks.

Chapter 14

BIBLIOGRAPHY

- [1] Al-Nafjan, A., Alharthi, K., & Kurdi, H. (2020). Lightweight Building of an Electroencephalogram-Based Emotion Detection System. *Brain Sciences*, 10(11), 781. MDPI.
- [2] Yao, L., Wang, M., Lu, Y., Li, H., & Zhang, X. (2021). EEG-Based Emotion Recognition by Exploiting Fused Network Entropy Measures of Complex Networks across Subjects. *Entropy*, 23(8), 984. MDPI+1.
- [3] Akter, S., Prodhan, R. A., Pias, T. S., Eisenberg, D., & Fernandez, J. F. (2022). M1M2: Deep-Learning-Based Real-Time Emotion Recognition from Neural Activity. *Sensors*, 22(21), 8467. MDPI.
- [4] Yin, Y., Zheng, X., Hu, B., Zhang, Y., & Cui, X. (2021). EEG Emotion Recognition Using Fusion Model of Graph Convolutional Neural Networks and LSTM. *Applied Soft Computing*, 100, 106954. MDPI.
- [5] Asghar, M. A., Khan, M. J., Shahid, H., Shorfuzzaman, M., Xiong, N. N., & Mehmood, R. M. (2021). Semi-Skipping Layered Gated Unit and Efficient Network: Hybrid Deep Feature Selection Method for Edge Computing in EEG-Based Emotion Classification. *IEEE Access*, 9, 13378–13389.
- [6] Cheng, J., Chen, M., Li, C., Liu, Y., Song, R., Liu, A., & Chen, X. (2021). Emotion Recognition from Multi-Channel EEG via Deep Forest. *IEEE Journal of Biomedical and Health Informatics*, 25, 453–464.
- [7] Zhang, X., Du, T., & Zhang, Z. (2021). EEG Emotion Recognition Based on Channel Attention for E-Healthcare Applications. In *Transactions on Petri Nets and Other Models of Concurrency X* (pp. 159–169). Springer.
- [8] Liu, Y., & Fu, Y. (2021). Emotion Recognition by Deeply Learned Multi-Channel Textual and EEG Features. *Future Generation Computer Systems*, 119, 1–6. MDPI.